PUBLIC API DOCUMENTATION

RAG Railway API Usage Guide

Authentication, conventions, endpoint matrix, detailed contracts, examples, and workflows

PRODUCTION API BASE

`https://web-production-3bf65.up.railway.app/api/v1`

Authentication	Client operations	Documented routes
Bearer API key	95 endpoints	97 method/path pairs

Clickable endpoint matrix

Select any path in the Endpoint Reference to jump to its detailed contract.

The matrix includes 95 client-facing operations and 2 internal RunPod worker exchanges.

Contents

This guide mirrors docs/api-usage.md. The table of contents and Endpoint Reference paths link to their destination pages.

Complete API usage and endpoint details 6

- Current Capabilities . . . 6
  - Public API And Internal Tools . . . 7
- Authentication . . . 7
- API Key Scopes . . . 7
- Request Conventions . . . 8
  - JSON Requests . . . 8
  - Multipart Uploads . . . 8
  - UUIDs . . . 8
  - Timestamps . . . 8
  - Pagination . . . 8
  - Idempotency . . . 9
  - Standard Error Format . . . 9
- Enum Values . . . 10
  - Document Statuses . . . 10
  - Processing Job Statuses . . . 10
  - Processing Depth . . . 10
  - Chunk Types . . . 10
  - Generated File Formats . . . 10
  - Model Task Keys . . . 10
- Endpoint Reference . . . 10
- Environment Setup Examples . . . 14
- Health . . . 14
  - GET /api/v1/health . . . 14
- Current API Key . . . 14
  - GET /api/v1/me . . . 14
- Silos . . . 14
  - GET /api/v1/silos . . . 14
  - POST /api/v1/silos . . . 15
  - GET /api/v1/silos/{siloId} . . . 16
- Silo Projects . . . 16
  - GET /api/v1/silos/{siloId}/projects . . . 16
  - POST /api/v1/silos/{siloId}/projects . . . 16
  - GET /api/v1/silos/{siloId}/projects/{projectId} . . . 17
  - DELETE /api/v1/silos/{siloId}/projects/{projectId} . . . 17
  - GET /api/v1/silos/{siloId}/projects/{projectId}/documents . . . 17
  - PUT /api/v1/silos/{siloId}/projects/{projectId}/documents . . . 17

Documents . . . 18
GET /api/v1/silos/{siloId}/documents . . . 18
POST /api/v1/silos/{siloId}/documents . . . 19
GET /api/v1/documents/{documentId} . . . 20
GET /api/v1/documents/{documentId}/status . . . 21
GET /api/v1/documents/{documentId}/ingestion-stats . . . 22
GET /api/v1/documents/{documentId}/chunks . . . 23
GET /api/v1/documents/{documentId}/text . . . 24
GET /api/v1/documents/{documentId}/download . . . 24
POST /api/v1/documents/{documentId}/download-ticket . . . 24
POST /api/v1/documents/{documentId}/markdown-download-ticket . . . 24
GET /api/v1/runpod/download/{jobId} . . . 25
POST /api/v1/runpod/results . . . 25
POST /api/v1/documents/{documentId}/reprocess . . . 25
POST /api/v1/documents/{documentId}/embed . . . 26
POST /api/v1/documents/{documentId}/cancel . . . 26
DELETE /api/v1/documents/{documentId} . . . 27
Ingestion Webhooks . . . 27
Reference Libraries . . . 28
GET /api/v1/libraries . . . 28
POST /api/v1/libraries . . . 28
GET /api/v1/libraries/{libraryId} . . . 28
PATCH /api/v1/libraries/{libraryId} . . . 28
DELETE /api/v1/libraries/{libraryId} . . . 28
GET /api/v1/libraries/{libraryId}/projects . . . 28
POST /api/v1/libraries/{libraryId}/projects . . . 28
GET /api/v1/libraries/{libraryId}/projects/{projectId} . . . 29
PATCH /api/v1/libraries/{libraryId}/projects/{projectId} . . . 29
DELETE /api/v1/libraries/{libraryId}/projects/{projectId} . . . 29
GET /api/v1/libraries/{libraryId}/projects/{projectId}/documents . . . 29
POST /api/v1/libraries/{libraryId}/projects/{projectId}/documents . . . 29
DELETE /api/v1/libraries/{libraryId}/projects/{projectId}/documents/{documentId} . . . 29
POST /api/v1/libraries/{libraryId}/projects/{projectId}/search . . . 29
GET /api/v1/libraries/{libraryId}/projects/{projectId}/access . . . 30
PUT /api/v1/libraries/{libraryId}/projects/{projectId}/access . . . 30
DELETE /api/v1/libraries/{libraryId}/projects/{projectId}/access/{siloId} . . . 30
Template Groups . . . 30
GET /api/v1/template-groups . . . 30
GET /api/v1/template-groups/{templateGroupId} . . . 30
PATCH /api/v1/template-groups/{templateGroupId} . . . 30
GET /api/v1/template-groups/{templateGroupId}/templates . . . 30
POST /api/v1/template-groups/{templateGroupId}/templates/{templateId}/versions/{versionId}/review-findings/{findingId}/ignore . . . 30
POST /api/v1/template-groups/{templateGroupId}/templates/{templateId}/versions/{versionId}/review-findings/{findingId}/implement . . . 31

DELETE /api/v1/template-groups/{templateGroupId}/templates/{templateId} . . . 31
GET /api/v1/template-groups/{templateGroupId}/silos . . . 31
PUT /api/v1/template-groups/{templateGroupId}/silos . . . 31
DELETE /api/v1/template-groups/{templateGroupId}/silos/{siloId} . . . 31
GET /api/v1/federated-datasets . . . 31
POST /api/v1/federated-datasets . . . 31
GET /api/v1/federated-datasets/{datasetId} . . . 31
PATCH /api/v1/federated-datasets/{datasetId} . . . 32
DELETE /api/v1/federated-datasets/{datasetId} . . . 32
GET /api/v1/federated-datasets/{datasetId}/silos . . . 32
PUT /api/v1/federated-datasets/{datasetId}/silos . . . 32
DELETE /api/v1/federated-datasets/{datasetId}/silos/{siloId} . . . 32
DELETE /api/v1/federated-datasets/{datasetId}/silos/{siloId}/entries . . . 32
GET /api/v1/silos/{siloId}/federated-datasets . . . 32
GET /api/v1/silos/{siloId}/library-tools . . . 32
PUT /api/v1/silos/{siloId}/library-tools . . . 32
GET /api/v1/silos/{siloId}/projects/{projectId}/library-tools . . . 32
PUT /api/v1/silos/{siloId}/projects/{projectId}/library-tools . . . 33
RAG Settings . . . 33
GET /api/v1/rag-settings . . . 33
PUT /api/v1/rag-settings . . . 33
PATCH /api/v1/rag-settings . . . 34
RAG Search . . . 34
POST /api/v1/silos/{siloId}/rag-search . . . 34
Ask . . . 37
POST /api/v1/silos/{siloId}/ask . . . 37
POST /api/v1/silos/{siloId}/ask/stream . . . 39
GET /api/v1/ask-runs . . . 41
DELETE /api/v1/ask-runs . . . 42
GET /api/v1/ask-runs/{runId} . . . 43
POST /api/v1/ask-runs/{runId}/cancel . . . 44
Extractions . . . 44
GET /api/v1/silos/{siloId}/extractions . . . 44
POST /api/v1/silos/{siloId}/extractions . . . 45
GET /api/v1/extractions/{extractionId} . . . 47
Ask and standalone Extraction model-status webhooks . . . 47
Tasks . . . 48
GET /api/v1/tasks . . . 48
POST /api/v1/tasks . . . 48
GET /api/v1/tasks/{taskId} . . . 49
PUT /api/v1/tasks/{taskId} . . . 49
DELETE /api/v1/tasks/{taskId} . . . 50
PUT /api/v1/tasks/{taskId}/items . . . 50

POST /api/v1/tasks/{taskId}/run . . . 50
Task-run webhook callbacks . . . 51
GET /api/v1/task-runs . . . 54
GET /api/v1/task-runs/{runId} . . . 54
POST /api/v1/task-runs/{runId}/cancel . . . 55
Generated Files . . . 55
GET /api/v1/silos/{siloId}/generated-files . . . 55
GET /api/v1/generated-files/{fileId}/json . . . 56
GET /api/v1/generated-files/{fileId}/download . . . 56
POST /api/v1/generated-files/{fileId}/download-ticket . . . 57
DELETE /api/v1/generated-files/{fileId} . . . 57
Typical Workflows . . . 57
Workflow 1: Create a Silo, Upload a Document, Wait Until Ready, Ask a Question . . . 57
Workflow 2: RAG Search Then Download a Source Document . . . 58
Workflow 3: Ask and Save the Answer as a Generated File . . . 58
Workflow 4: Upload With a Webhook Callback . . . 59
Bubble API Connector Notes . . . 59
Operational Notes . . . 60
GET /api/v1/usage-logs . . . 60
Idempotency Storage . . . 60
Security . . . 61
Sleep/Wake Behavior . . . 61
Current Limitations . . . 61
Minimal Client Example . . . 61

Complete API usage and endpoint details

Last reviewed against the implemented `/api/v1` route handlers: 2026-08-15.

This guide is the canonical contract for the current Railway-hosted public API. It documents all public method/path combinations for creating silos and projects, ingesting documents, checking ingestion status, searching RAG content, asking grounded questions, executing extractions, running saved Tasks, retrieving generated files, and reading API diagnostics.

The implemented route tree currently contains 97 documented method/path combinations: 95 client-facing operations and 2 internal RunPod worker exchanges. The Endpoint Reference and the detailed endpoint headings are checked automatically against the route handlers so additions, removals, or stale paths fail the documentation validation step.

The current public API prefix is:

```
/api/v1
```

The current Railway production base URL is:

```
https://web-production-3bf65.up.railway.app
```

Use the full API base URL:

```
https://web-production-3bf65.up.railway.app/api/v1
```

For local development, use:

```
http://127.0.0.1:3000/api/v1
```

Current Capabilities

The public API currently supports:

- API-key authenticated requests.
- API-key scopes for separating read, ingest, delete, cancel, reprocess, ask, extract, and generated-file access.
- Standard JSON error responses.
- Request IDs on JSON responses.
- Pagination on list endpoints.
- Idempotency keys on state-changing create, upload, update, delete, cancel, reprocess, Ask, Extraction, Task, library, and internal callback operations.
- Usage audit logging per API key in `api_usage_logs`.
- API-key-scoped usage-log retrieval for Bubble diagnostics.
- Optional ingestion/reprocess webhook callbacks.
- Silo-scoped document ingestion.
- Document status, chunks, original downloads, and Markdown text downloads.
- Short-lived signed download tickets for originals, canonical Markdown, and generated files.
- RAG search with source document IDs and full source document names.
- Ask calls against selected silo/document RAG content, including a bounded model tool loop.
- NDJSON Ask streaming with reasoning summaries/traces, model output, tool lifecycle events, and a final result event.
- Durable Ask run status and event recovery after a client disconnect.
- Generated Ask output files when the Ask task has the `write_file` tool enabled.
- Synchronous standalone Extraction execution with persisted run, result, and generated-file records.
- Saved extraction Tasks that run multiple extraction outputs sequentially against a selected silo.

- Generated-file listing and download.
- API-key-scoped reference-library creation, ingestion, search, access grants, and Ask/Extraction tool bindings.

Public API And Internal Tools

Client-callable routes use the `/api/v1` prefix. The two RunPod routes listed in this guide are internal worker exchanges under the same prefix and accept short-lived job credentials rather than administrator-created API keys. The Railway admin UI also uses private `/api/...` routes, and the model runtime has internal tools such as `rag_search`, `list_files`, `get_document_markdown`, `get_document_image`, `render_pdf_page`, and `write_file`. Those private admin routes, internal RunPod exchanges, and model-tool arguments must not be called directly from Bubble or another external client.

When an external client needs a complete ingested document in Markdown, use `GET /api/v1/documents/{documentId}/text` or create a browser-safe URL with `POST /api/v1/documents/{documentId}/markdown-download-ticket`. The internal `get_document_markdown` tool gives the Ask or Extraction model exact paged ranges of that same canonical Markdown while enforcing the selected tenant, silo, and optional project scope.

Authentication

All client-callable `/api/v1` routes require an API key except:

```
GET /api/v1/health
```

`GET /api/v1/runpod/download/{jobId}` and `POST /api/v1/runpod/results` are not client-callable exceptions. They require the short-lived bearer credential issued for one RunPod embedding attempt; normal `brk_` API keys are rejected.

Send the API key as a bearer token:

```
Authorization: Bearer brk_your_api_key_here
```

Example:

```
curl -sS "$API_BASE/me" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

API keys are created under Railway admin **Configuration -> API functionality -> API keys**. The key is generated by the backend and starts with `brk_`.

API Key Scopes

Each API key may have one or more scopes.

Scope	Allows
`silos:read`	List and fetch silos, and read RAG ingestion/search settings.
`silos:write`	Create silos and update RAG ingestion/search settings.
`documents:read`	List documents, fetch document metadata/status/chunks/text/downloads, and run RAG search.
`documents:ingest`	Upload documents into a silo and queue ingestion.
`documents:delete`	Delete a document and its related stored objects.
`documents:reprocess`	Queue reprocessing for an existing document.
`documents:cancel`	Cancel an active queued or running document ingestion/reprocess job.
`rag:ask`	Ask questions against RAG content.
`rag:extract`	Execute/fetch standalone extractions and manage saved multi-step extraction Tasks/runs.
`generated-files:read`	List and download generated files.
`generated-files:delete`	Delete generated files and their stored objects.
`api-usage:read`	Read redacted API usage logs for the current key or tenant.
`libraries:read`	List/search libraries and read their projects, documents, status, and downloads.

Scope	Allows
`libraries:write`	Create/edit libraries and projects, upload files, and change document membership.
`libraries:grant`	Grant library projects to silos and bind their tools to Ask or Extraction.

The `/api/v1/me` endpoint only requires a valid API key and returns the key identity and scopes.

Administrators assign the three library permissions under **Configuration -> API functionality -> API keys -> Reference library access**. Select the permissions on an existing key and choose **Save library access**. An API username inherits the scopes of its assigned API key, so an API user sees the Reference Libraries interface only when that key includes `libraries:read`. The `libraries:grant` scope is deliberately separate because it can expose a library project's RAG tool to a silo's Ask or Extraction workflow.

Request Conventions

JSON Requests

Most POST endpoints use JSON:

```
Content-Type: application/json
```

Multipart Uploads

Document ingestion uses multipart form data and accepts either a binary file part or a public HTTPS URL source:

```
Content-Type: multipart/form-data
```

- `file`: a real binary multipart file part. A URL string in this legacy field is also accepted for Bubble compatibility.
- `fileUrl`: the preferred field when Bubble's File Uploader provides an HTTPS URL instead of raw bytes.

At least one source is required. Do not set the multipart boundary manually; Bubble and normal HTTP clients generate it. Send `fileName` with URL uploads so the parser receives the correct extension.

UUIDs

Most object IDs are UUID strings:

```
1fb52fc2-b5ce-4853-b9a6-294ab95429ca
```

Timestamps

Timestamps are returned as ISO 8601 strings.

Pagination

List endpoints accept:

Query param	Default	Min	Max
`page`	`1`	`1`	`10000`
`pageSize`	`20`	`1`	`100`

Paginated JSON responses include:

```
{
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 48,
    "totalPages": 3
  }
}
```

Idempotency

Endpoints that create or mutate persistent state require:

```
Idempotency-Key: unique-client-generated-key
```

The key can be any stable string up to 160 characters. A UUID is a good choice.

If the same API key sends the same idempotency key with the same request shape, the API replays the saved JSON response. If the request shape differs, the API returns:

```
{
  "error": {
    "code": "idempotency_key_conflict",
    "message": "This Idempotency-Key was already used for a different request shape.",
    "details": null
  },
  "requestId": "..."
}
```

For the streaming Ask endpoint, an idempotency replay does not start a second model run. It returns a short `ask.replayed` NDJSON stream containing the original `runId` and `statusUrl`. Use that URL to retrieve the durable state.

Replayed responses include the `Idempotency-Replayed: true` header. API usage logs also record `requestMetadata.idempotencyReplayed: true`.

Use idempotency keys for retries after network failures or client timeouts. Generate a new key for every new logical operation, even when the new request has the same URL and JSON body as an earlier operation.

If a project was created, later deleted, and a client reuses the original project-create idempotency key, the API returns `409` with:

```
{
  "error": {
    "code": "idempotency_replay_resource_deleted",
    "message": "This Idempotency-Key belongs to a project that has since been deleted. Send a
      new unique Idempotency-Key to create another project."
  },
  "requestId": "..."
}
```

Multipart upload hashes are canonicalized from the logical form fields, file metadata, and file bytes. The multipart boundary and field ordering do not affect the hash, so Bubble can safely retry the same upload with the same idempotency key even when it regenerates the multipart body.

Standard Error Format

JSON API errors use:

```
{
  "error": {
    "code": "invalid_request",
    "message": "Request body did not match the API contract.",
    "details": {}
  },
  "requestId": "..."
}
```

Common status codes:

Status	Meaning
`400`	Invalid JSON, missing idempotency key, or request hash problem.
`401`	Missing or invalid API key.
`403`	API key is valid but does not have the required scope.
`404`	Requested silo, document, file, or extraction was not found.
`409`	Duplicate resource or idempotency-key conflict.
`422`	Request body or query/form fields are invalid.
`500`	Server/database/storage failure.

Status	Meaning
503	Required runtime configuration is missing.

Enum Values

Document Statuses

```
uploaded
queued
extracting_text
ocr_running
vision_running
embedding
ready
failed
deleted
```

Processing Job Statuses

```
queued
running
succeeded
failed
cancelled
```

Processing Depth

```
fast
standard
deep
```

Chunk Types

```
native_text
ocr_text
table
figure
chart
diagram
screenshot
form_region
page_summary
```

Generated File Formats

```
txt
csv
md
json
```

Model Task Keys

```
vision
ask
extraction
pdf_assembly
```

Endpoint Reference

The generated `api-usage.pdf` makes every method/path entry in this matrix an internal link to that endpoint's detailed section.

Method	Path	Scope	Idempotency	Purpose
GET	/api/v1/health	None	No	Public API health check.
GET	/api/v1/me	Any valid key	No	Inspect current API key and tenant.
GET	/api/v1/silos	silos:read	No	List silos.
POST	/api/v1/silos	silos:write	Required	Create a silo.

Method	Path	Scope	Idempotency	Purpose
GET	/api/v1/silos/{siloId}	silos:read	No	Fetch one silo.
GET	/api/v1/silos/{siloId}/projects	silos:read	No	List projects/subsets inside a silo.
POST	/api/v1/silos/{siloId}/projects	silos:write	Required	Create a project/subset inside a silo.
GET	/api/v1/silos/{siloId}/projects/{projectId}	silos:read	No	Fetch one project/subset inside a silo.
DELETE	/api/v1/silos/{siloId}/projects/{projectId}	silos:write	Required	Delete one project/subset; documents and stored files are kept.
GET	/api/v1/silos/{siloId}/projects/{projectId}/documents	documents:read	No	List documents assigned to one project/subset.
PUT	/api/v1/silos/{siloId}/projects/{projectId}/documents	documents:ingest	Required	Replace the document membership list for one project/subset.
GET	/api/v1/libraries	libraries:read	No	List tenant reference libraries.
POST	/api/v1/libraries	libraries:write	Required	Create a reference library.
GET	/api/v1/libraries/{libraryId}	libraries:read	No	Fetch one reference library.
PATCH	/api/v1/libraries/{libraryId}	libraries:write	Required	Rename or re-describe a library.
DELETE	/api/v1/libraries/{libraryId}	libraries:write	Required	Delete a library and its projects; source documents are retained.
GET	/api/v1/libraries/{libraryId}/projects	libraries:read	No	List the projects in a library.
POST	/api/v1/libraries/{libraryId}/projects	libraries:write	Required	Create a library project and model tool definition.
GET	/api/v1/libraries/{libraryId}/projects/{projectId}	libraries:read	No	Fetch one library project.
PATCH	/api/v1/libraries/{libraryId}/projects/{projectId}	libraries:write	Required	Edit a library project and model tool definition.
DELETE	/api/v1/libraries/{libraryId}/projects/{projectId}	libraries:write	Required	Delete a project and memberships; source documents are retained.
GET	/api/v1/libraries/{libraryId}/projects/{projectId}/documents	libraries:read	No	List documents assigned to a library project.
POST	/api/v1/libraries/{libraryId}/projects/{projectId}/documents	libraries:write	Required	Upload and queue a document, or assign an existing library document.
DELETE	/api/v1/libraries/{libraryId}/projects/{projectId}/documents/{documentId}	libraries:write	Required	Remove project membership without deleting the document.
POST	/api/v1/libraries/{libraryId}/projects/{projectId}/search	libraries:read	No	Search only the selected library project's ready documents.
GET	/api/v1/libraries/{libraryId}/projects/{projectId}/access	libraries:grant	No	List silos granted access to a library project.
PUT	/api/v1/libraries/{libraryId}/projects/{projectId}/access	libraries:grant	Required	Replace the complete silo-access list.
DELETE	/api/v1/libraries/{libraryId}/projects/{projectId}/access/{siloId}	libraries:grant	Required	Revoke one silo's access.
GET	/api/v1/template-groups	templates:read	No	List Template Groups and their IDs.
GET	/api/v1/template-groups/{templateGroupId}	templates:read	No	Fetch one Template Group and model tool definition.
PATCH	/api/v1/template-groups/{templateGroupId}	templates:write	Required	Edit the group name, tool name, description, or enabled state.
GET	/api/v1/template-groups/{templateGroupId}/templates	templates:read	No	List active templates and their IDs inside one Template Group.
DELETE	/api/v1/template-groups/{templateGroupId}/templates/{templateId}	templates:write	Required	Remove one template from the catalog while retaining private history.
POST	/api/v1/template-groups/{templateGroupId}/templates/{templateId}/versions/{versionId}/review-findings/{findingId}/ignore	templates:write	No	Ignore one finding for the analyzed version.

Method	Path	Scope	Idempotency	Purpose
POST	/api/v1/template-groups/{templateGroupId}/templates/{templateId}/versions/{versionId}/review-findings/{findingId}/implement	templates:write	Required	Create and re-ingest a corrected immutable DOCX version.
GET	/api/v1/template-groups/{templateGroupId}/silos	templates:grant	No	List silos assigned to a Template Group.
PUT	/api/v1/template-groups/{templateGroupId}/silos	templates:grant	Required	Replace the complete silo-assignment list.
DELETE	/api/v1/template-groups/{templateGroupId}/silos/{siloId}	templates:grant	Required	Remove one Template Group assignment from a silo.
GET	/api/v1/federated-datasets	federated:read	No	List Federated Datasets and their IDs.
POST	/api/v1/federated-datasets	federated:write	Required	Create a Federated Dataset from a name, tool name, and description.
GET	/api/v1/federated-datasets/{datasetId}	federated:read	No	Fetch one Federated Dataset and its model tool definition.
PATCH	/api/v1/federated-datasets/{datasetId}	federated:write	Required	Edit the dataset name, tool name, description, or model write switches.
DELETE	/api/v1/federated-datasets/{datasetId}	federated:write	Required	Delete the dataset and every silo's entries. Not recoverable.
GET	/api/v1/federated-datasets/{datasetId}/silos	federated:grant	No	List silos the Federated Dataset is assigned to.
PUT	/api/v1/federated-datasets/{datasetId}/silos	federated:grant	Required	Replace the complete silo-assignment list. Omitted silos keep their data.
DELETE	/api/v1/federated-datasets/{datasetId}/silos/{siloId}	federated:grant	Required	Unassign one silo. Its entries are retained.
DELETE	/api/v1/federated-datasets/{datasetId}/silos/{siloId}/entries	federated:write	Required	Clear one silo's entries for the dataset. Destructive.
GET	/api/v1/silos/{siloId}/federated-datasets	federated:read	No	List Federated Datasets available to one silo.
GET	/api/v1/silos/{siloId}/library-tools	libraries:grant	No	List silo-default Ask/Extraction library bindings.
PUT	/api/v1/silos/{siloId}/library-tools	libraries:grant	Required	Replace silo-default library bindings for one task type.
GET	/api/v1/silos/{siloId}/projects/{projectId}/library-tools	libraries:grant	No	List project-specific library bindings.
PUT	/api/v1/silos/{siloId}/projects/{projectId}/library-tools	libraries:grant	Required	Replace project-specific library bindings for one task type.
GET	/api/v1/rag-settings	silos:read	No	Read chunking, embedding model, and direct-search defaults.
PUT	/api/v1/rag-settings	silos:write	Required	Update chunking, embedding model, and direct-search defaults.
PATCH	/api/v1/rag-settings	silos:write	Required	Update direct-search defaults without changing embedding or chunking.
GET	/api/v1/silos/{siloId}/documents	documents:read	No	List documents in a silo.
POST	/api/v1/silos/{siloId}/documents	documents:ingest	Required	Upload and queue a document for ingestion.
GET	/api/v1/documents/{documentId}	documents:read	No	Fetch document metadata.
DELETE	/api/v1/documents/{documentId}	documents:delete	Required	Delete document and related stored objects.
GET	/api/v1/documents/{documentId}/status	documents:read	No	Fetch document, latest job heartbeat/timing, chunk/embedding/asset counts.
GET	/api/v1/documents/{documentId}/ingestion-stats	documents:read	No	Fetch provider call outcomes, timing, and pages-per-call statistics.
GET	/api/v1/documents/{documentId}/chunks	documents:read	No	List stored chunks for a document.
GET	/api/v1/documents/{documentId}/text	documents:read	No	Download document text as Markdown.
GET	/api/v1/documents/{documentId}/download	documents:read	No	Download original uploaded source file.
POST	/api/v1/documents/{documentId}/download-ticket	documents:read	No	Create a short-lived original-file download URL.

Method	Path	Scope	Idempotency	Purpose
POST	/api/v1/documents/{documentId}/markdown-download-ticket	documents:read	No	Create a short-lived canonical Markdown download URL.
GET	/api/v1/runpod/download/{jobId}	RunPod job token	No	Internal one-attempt canonical Markdown download.
POST	/api/v1/runpod/results	RunPod job token	Required	Internal idempotent accelerator result callback.
POST	/api/v1/documents/{documentId}/reprocess	documents:reprocess	Required	Queue reprocessing.
POST	/api/v1/documents/{documentId}/embed	documents:reprocess	Required	Queue embedding-only vector backfill for a ready document.
POST	/api/v1/documents/{documentId}/cancel	documents:cancel	Required	Cancel the latest active document processing job, or a supplied job ID.
POST	/api/v1/silos/{siloId}/rag-search	documents:read	No	Search indexed chunks in a silo.
POST	/api/v1/silos/{siloId}/ask	rag:ask	Required	Ask a grounded question against RAG content.
POST	/api/v1/silos/{siloId}/ask/stream	rag:ask	Required	Stream a durable Ask run as newline-delimited JSON events.
GET	/api/v1/ask-runs	rag:ask	No	List and filter persisted Ask runs in a silo or project.
DELETE	/api/v1/ask-runs	rag:ask	Required	Clear terminal Ask runs in a silo or project; active runs are retained.
GET	/api/v1/ask-runs/{runId}	rag:ask	No	Retrieve Ask status, final result, files, and persisted events.
POST	/api/v1/ask-runs/{runId}/cancel	rag:ask	Required	Cancel an Ask and all active swarm workers.
GET	/api/v1/silos/{siloId}/extractions	rag:extract	No	List extraction runs in a silo.
POST	/api/v1/silos/{siloId}/extractions	rag:extract	Required	Execute and persist a standalone extraction.
GET	/api/v1/extractions/{extractionId}	rag:extract	No	Fetch extraction run, result, and generated files.
GET	/api/v1/tasks	rag:extract	No	List saved multi-step extraction Tasks.
POST	/api/v1/tasks	rag:extract	Required	Create a saved extraction Task definition.
GET	/api/v1/tasks/{taskId}	rag:extract	No	Fetch one saved Task definition.
PUT	/api/v1/tasks/{taskId}	rag:extract	Required	Update saved Task name, description, active state, and optionally items.
PUT	/api/v1/tasks/{taskId}/items	rag:extract	Required	Replace the ordered extraction-item sequence for a saved Task.
DELETE	/api/v1/tasks/{taskId}	rag:extract	Required	Delete a saved Task definition.
POST	/api/v1/tasks/{taskId}/run	rag:extract	Required	Run a saved Task sequentially against a selected silo.
GET	/api/v1/task-runs	rag:extract	No	List silo-specific Task runs and progress.
GET	/api/v1/task-runs/{runId}	rag:extract	No	Fetch Task run detail and ordered item status.
POST	/api/v1/task-runs/{runId}/cancel	rag:extract	Required	Cancel a queued/running Task run.
GET	/api/v1/silos/{siloId}/generated-files	generated-files:read	No	List generated files in a silo.
GET	/api/v1/generated-files/{fileId}/json	generated-files:read	No	Return a generated JSON file as parsed JSON.
GET	/api/v1/generated-files/{fileId}/download	generated-files:read	No	Download one generated file.
POST	/api/v1/generated-files/{fileId}/download-ticket	generated-files:read	No	Create a short-lived generated-file download URL.
DELETE	/api/v1/generated-files/{fileId}	generated-files:delete	Required	Delete one generated file and its stored object.
GET	/api/v1/usage-logs	api-usage:read	No	List redacted, paginated API usage logs.

Environment Setup Examples

```
export API_BASE="https://web-production-3bf65.up.railway.app/api/v1"
export BUBBLE_RAG_API_KEY="brk_replace_me"
```

PowerShell:

```
$env:API_BASE = "https://web-production-3bf65.up.railway.app/api/v1"
$env:BUBBLE_RAG_API_KEY = "brk_replace_me"
```

Health

`GET /api/v1/health`

Public endpoint. No API key required.

Request:

```
curl -sS "$API_BASE/health"
```

Response:

```
{
  "ok": true,
  "version": "v1"
}
```

Current API Key

`GET /api/v1/me`

Returns the authenticated API key summary and tenant ID.

Request:

```
curl -sS "$API_BASE/me" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "apiKey": {
    "id": "api-key-uuid",
    "name": "Bubble Connector",
    "scopes": ["silos:read", "documents:read", "rag:ask"]
  },
  "tenantId": "00000000-0000-4000-8000-000000000001",
  "requestId": "..."
}
```

Silos

Silos partition the RAG database. Uploads, RAG search, Ask, extraction, and generated files are scoped to a silo.

`GET /api/v1/silos`

Scope: `silos:read`

Query params:

Name	Type	Notes
`page`	integer	Optional pagination page.
`pageSize`	integer	Optional pagination size.

Request:

```
curl -sS "$API_BASE/silos?page=1&pageSize=20" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "silos": [
    {
      "id": "00000000-0000-4000-8000-000000000401",
      "tenant_id": "00000000-0000-4000-8000-000000000001",
      "project_id": "00000000-0000-4000-8000-000000000101",
      "name": "Default",
      "slug": "default",
      "description": null,
      "created_at": "2026-07-19T00:00:00.000Z",
      "updated_at": "2026-07-19T00:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 1,
    "totalPages": 1
  },
  "requestId": "..."
}
```

`POST /api/v1/silos`

Scope: `silos:write`

Idempotency required.

Standalone extractions and sequential Task steps use the saved Extraction model task policy. Railway presents the extraction request to the model before RAG. The model can call scoped `rag_search` when evidence is needed, up to **Max RAG searches per request** (`0-100`, default `24`). **Max tool requests** is a separate overall ceiling (`1-250`, default `120`) so multi-step file construction does not compete with a mandatory search minimum. **Max request characters** applies to the Extraction request itself (`4,000-1,000,000`, default `100,000`). Usage records the configured request limit and the actual RAG call count.

Body:

Field	Type	Required	Notes
`name`	string	Yes	1-120 characters.
`description`	string	No	Up to 1000 characters.
`projectId`	UUID	No	Defaults to the default project ID.

Request:

```
curl -sS "$API_BASE/silos" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: silo-create-$(uuidgen)" \
  -d '{
    "name": "Client A Contracts",
    "description": "Contracts and supporting documents for Client A."
  }'
```

Response status: `201`

```
{
  "silo": {
    "id": "silo-uuid",
    "tenant_id": "tenant-uuid",
    "project_id": "project-uuid",
    "name": "Client A Contracts",
    "slug": "client-a-contracts",
    "description": "Contracts and supporting documents for Client A.",
    "created_at": "2026-07-19T00:00:00.000Z",
    "updated_at": "2026-07-19T00:00:00.000Z"
  },
  "requestId": "..."
}
```

`GET /api/v1/silos/{siloId}`

Scope: `silos:read`

Request:

```
curl -sS "$API_BASE/silos/$SILO_ID" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "silo": {
    "id": "silo-uuid",
    "tenant_id": "tenant-uuid",
    "project_id": "project-uuid",
    "name": "Client A Contracts",
    "slug": "client-a-contracts",
    "description": null,
    "created_at": "2026-07-19T00:00:00.000Z",
    "updated_at": "2026-07-19T00:00:00.000Z"
  },
  "requestId": "..."
}
```

Silo Projects

Projects are optional sub-scopes inside a silo. A silo contains the full document set; a project contains a subset of those documents. One document can belong to multiple projects without being uploaded or ingested more than once.

When `projectId` is omitted from search, Ask, Extraction, Task run, generated-file list, or document-list requests, the API uses the whole silo. When `projectId` is supplied, the API limits retrieval/listing to documents assigned to that project.

To add existing ingested files to a project without re-ingesting them:

1. List the whole silo with `GET /api/v1/silos/{siloId}/documents` and no `projectId`.
2. List the current project membership with `GET /api/v1/silos/{siloId}/projects/{projectId}/documents`.
3. Send the complete desired membership list to `PUT /api/v1/silos/{siloId}/projects/{projectId}/documents`.

`GET /api/v1/silos/{siloId}/projects`

Scope: `silos:read`

Lists projects in a silo. Supports `page` and `pageSize`.

```
curl -sS "$API_BASE/silos/$SILO_ID/projects?page=1&pageSize=20" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "projects": [
    {
      "id": "project-uuid",
      "tenant_id": "tenant-uuid",
      "silo_id": "silo-uuid",
      "name": "Sidewalk Projects",
      "slug": "sidewalk-projects",
      "description": "Subset used for sidewalk-related extractions.",
      "created_at": "2026-08-01T12:00:00.000Z",
      "updated_at": "2026-08-01T12:00:00.000Z"
    }
  ],
  "pagination": { "page": 1, "pageSize": 20, "total": 1, "totalPages": 1 },
  "requestId": "..."
}
```

`POST /api/v1/silos/{siloId}/projects`

Scope: `silos:write`

Idempotency required.

```
curl -sS "$API_BASE/silos/$SILO_ID/projects" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: project-$(uuidgen)" \
  -d '{
    "name": "Sidewalk Projects",
    "description": "Subset used for sidewalk-related extractions."
  }'
```

GET /api/v1/silos/{siloId}/projects/{projectId}

Scope: `silos:read`

Returns one project only when it belongs to both the authenticated tenant and the selected silo.

```
curl -sS "$API_BASE/silos/$SILO_ID/projects/$PROJECT_ID" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

The response is `{ "project": { ... }, "requestId": "..." }` using the same project fields returned by the project list.

DELETE /api/v1/silos/{siloId}/projects/{projectId}

Scope: `silos:write`

Idempotency required.

Deletes only the project/subset and its membership rows. Uploaded documents, original stored files, generated files, chunks, and embeddings are not deleted.

```
curl -sS "$API_BASE/silos/$SILO_ID/projects/$PROJECT_ID" \
  -X DELETE \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: delete-project-$(uuidgen)"
```

Response:

```
{
  "deleted": true,
  "project": { "id": "project-uuid", "name": "Sidewalk Projects" },
  "documentsDeleted": 0,
  "note": "Only the project and its document memberships were removed.",
  "requestId": "..."
}
```

GET /api/v1/silos/{siloId}/projects/{projectId}/documents

Scope: `documents:read`

Lists the documents currently assigned to the project. Use this to pre-check document assignment controls before replacing membership.

Supports `page` and `pageSize`. The response contains `project`, the complete `documentIds` membership array, the current paginated `documents` page with download links, `pagination`, and `requestId`.

```
curl -sS "$API_BASE/silos/$SILO_ID/projects/$PROJECT_ID/documents?page=1&pageSize=20" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

PUT /api/v1/silos/{siloId}/projects/{projectId}/documents

Scope: `documents:ingest`

Idempotency required.

Replaces the project membership list with document IDs that already exist in the selected silo.

```
curl -sS "$API_BASE/silos/$SILO_ID/projects/$PROJECT_ID/documents" \
  -X PUT \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: project-docs-$(uuidgen)" \
  -d '{
    "documentIds": ["document-uuid-1", "document-uuid-2"]
  }'
```

Documents

Documents are original files uploaded into a silo. A document becomes searchable after the worker processes it and its status becomes `ready`.

Document list, upload, detail, status, and reprocess responses include a `downloads` object on each document:

```
{
  "downloads": {
    "original": {
      "method": "GET",
      "url": "/api/v1/documents/document-uuid/download",
      "ticketMethod": "POST",
      "ticketUrl": "/api/v1/documents/document-uuid/download-ticket"
    },
    "markdown": {
      "method": "GET",
      "url": "/api/v1/documents/document-uuid/text",
      "contentType": "text/markdown; charset=utf-8",
      "ticketMethod": "POST",
      "ticketUrl": "/api/v1/documents/document-uuid/markdown-download-ticket"
    }
  }
}
```

Use `downloads.original.url` to stream the exact uploaded source file through Railway. Use `downloads.markdown.url` to stream the text/Markdown version created during ingestion. Bubble browser workflows should call the corresponding `ticketUrl`, then open the returned short-lived URL without exposing the API key. If no stored source-assembly Markdown exists yet, the Markdown ticket endpoint builds and privately caches canonical Markdown from the stored pages/chunks. Excel downloads return the sheet-by-sheet Markdown table assembly. PDFs and DOCX files with embedded-image analysis enabled include image-analysis sections for eligible extracted images, including vision descriptions, transcribed visible text, table conversions, and chart/value estimates where possible.

`GET /api/v1/silos/{siloId}/documents`

Scope: `documents:read`

Query params:

Name	Type	Notes
`page`	integer	Optional pagination page.
`pageSize`	integer	Optional pagination size.
`projectId`	UUID	Optional project/subset filter inside this silo.
`status`	string	Optional status filter, such as `ready`, `queued`, or `failed`.

Request:

```
curl -sS "$API_BASE/silos/$SILO_ID/documents?status=ready&page=1&pageSize=20" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "documents": [
    {
      "id": "document-uuid",
      "tenant_id": "tenant-uuid",
      "project_id": "project-uuid",
      "silo_id": "silo-uuid",
      "file_name": "source.pdf",
      "title": "source.pdf",
      "status": "ready",
      "page_count": 12,
      "mime_type": "application/pdf",
      "processing_depth": "standard",
      "parser_provider": "worker",
      "parser_version": "v1",
      "last_error": null,
      "created_at": "2026-07-19T00:00:00.000Z",
      "updated_at": "2026-07-19T00:05:00.000Z",
      "downloads": {
        "original": {
```

```
          "method": "GET",
          "url": "/api/v1/documents/document-uuid/download"
        },
        "markdown": {
          "method": "GET",
          "url": "/api/v1/documents/document-uuid/text",
          "contentType": "text/markdown; charset=utf-8"
        }
      }
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 1,
    "totalPages": 1
  },
  "requestId": "..."
}
```

`POST /api/v1/silos/{siloId}/documents`

Scope: `documents:ingest`

Idempotency required.

Uploads a file, creates a document record, creates a source `document_files` row, queues an ingestion job, and sets the document status to `queued`.

Supported source uploads currently include PDF, DOCX, Excel workbooks, Markdown, plain text, CSV, PNG, and JPEG. PDF, DOCX, and Excel sources are first assembled into a canonical Markdown file before chunking and embedding. Excel sheets are emitted as Markdown table sections with row numbers and Excel column letters preserved. DOCX embedded PNG/JPEG images are extracted as document assets and, when DOCX vision analysis is enabled, inserted back into the Markdown as vision-generated image analysis blocks. Markdown, text, and CSV files use the same canonical Markdown -> chunk -> embed path without a separate model call.

Form fields:

Field	Type	Required	Notes
`file`	file or HTTPS URL string	Conditional	Original source bytes. A URL string is accepted for compatibility with Bubble's File Uploader value.
`fileUrl`	HTTPS URL string	Conditional	Preferred Bubble integration field. Railway downloads the Bubble-hosted file after public-network and size validation. Send either `file` or `fileUrl`.
`fileName`	string	No	Overrides uploaded or remote file name. Send this for `fileUrl` uploads so the source extension is preserved.
`title`	string	No	Defaults to file name.
`mimeType`	string	No	Defaults to uploaded file MIME type. If omitted or `application/octet-stream`, the API infers common source types from the file extension, including `.docx`, `.xlsx`, `.xls`, `.xlsm`, `.xlsb`, `.md`, `.txt`, `.csv`, `.pdf`, `.png`, and `.jpg`.
`processingDepth`	`fast` | `standard` | `deep`	No	Defaults to `standard`.
`projectId`	UUID	No	Assigns the uploaded document to this project/subset after the single document record is created. The document remains part of the main silo and may also be assigned to other projects.
`webhookUrl`	URL	No	Receives completion/failure callbacks. Must be HTTP or HTTPS.
`webhookSecret`	string	No	Used to sign webhook payloads with HMAC-SHA256.

Request:

```
curl -sS "$API_BASE/silos/$SILO_ID/documents" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: ingest-$(uuidgen)" \
  -F "file=@./source.pdf" \
  -F "title=Source Agreement" \
  -F "processingDepth=standard" \
  -F "webhookUrl=https://example.com/bubble-rag/webhook" \
```

```
  -F "webhookSecret=replace-with-your-shared-secret"
```

Bubble File Uploader request:

```
Body type: Form-data
fileUrl = File Uploader A's value
fileName = File Uploader A's value's file name
processingDepth = standard
projectId = selected Railway project UUID (omit when blank)
```

Do not set the multipart `Content-Type` manually. Bubble creates the boundary. The File Uploader's value must be non-empty before this action runs. Remote URL downloads accept HTTPS only, reject private/local network destinations and unsafe redirects, and default to a 100 MB maximum controlled by `API_REMOTE_UPLOAD_MAX_BYTES`.

Response status: `202`


```
{
  "document": {
    "id": "document-uuid",
    "tenant_id": "tenant-uuid",
    "project_id": "project-uuid",
    "silo_id": "silo-uuid",
    "file_name": "source.pdf",
    "title": "Source Agreement",
    "status": "queued",
    "created_at": "2026-07-19T00:00:00.000Z",
    "downloads": {
      "original": {
        "method": "GET",
        "url": "/api/v1/documents/document-uuid/download"
      },
      "markdown": {
        "method": "GET",
        "url": "/api/v1/documents/document-uuid/text",
        "contentType": "text/markdown; charset=utf-8"
      }
    }
  },
  "job": {
    "id": "job-uuid",
    "document_id": "document-uuid",
    "status": "queued",
    "stage": "registered",
    "queued_at": "2026-07-19T00:00:00.000Z"
  },
  "file": {
    "bucket": "original-documents",
    "storagePath": "documents/tenant-uuid/document-uuid/source.pdf",
    "byteSize": 123456,
    "sha256": "hex-sha256"
  },
  "requestId": "..."
}
```


`GET /api/v1/documents/{documentId}`

Scope: `documents:read`

Fetches document metadata.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:


```
{
  "document": {
    "id": "document-uuid",
    "tenant_id": "tenant-uuid",
    "project_id": "project-uuid",
    "silo_id": "silo-uuid",
    "file_name": "source.pdf",
    "title": "Source Agreement",
    "status": "ready",
    "page_count": 12,
    "mime_type": "application/pdf",
    "processing_depth": "standard",
    "parser_provider": "worker",
```

```
    "parser_version": "v1",
    "last_error": null,
    "created_at": "2026-07-19T00:00:00.000Z",
    "updated_at": "2026-07-19T00:05:00.000Z",
    "downloads": {
      "original": {
        "method": "GET",
        "url": "/api/v1/documents/document-uuid/download"
      },
      "markdown": {
        "method": "GET",
        "url": "/api/v1/documents/document-uuid/text",
        "contentType": "text/markdown; charset=utf-8"
      }
    }
  },
  "requestId": "..."
}
```


GET /api/v1/documents/{documentId}/status

Scope: `documents:read`

Returns document metadata, latest processing job, heartbeat/timing details, chunk count, and asset count. Poll this endpoint while `latestJob.status` is `queued` or `running`; `latestJob.timing.isStale` becomes `true` when a running worker has not refreshed its heartbeat recently.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/status" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:


```
{
  "document": {
    "id": "document-uuid",
    "status": "ready",
    "file_name": "source.pdf",
    "last_error": null,
    "downloads": {
      "original": {
        "method": "GET",
        "url": "/api/v1/documents/document-uuid/download"
      },
      "markdown": {
        "method": "GET",
        "url": "/api/v1/documents/document-uuid/text",
        "contentType": "text/markdown; charset=utf-8"
      }
    }
  },
  "latestJob": {
    "id": "job-uuid",
    "status": "succeeded",
    "stage": "complete",
    "attempts": 1,
    "error_message": null,
    "locked_at": "2026-07-23T16:13:14.000Z",
    "locked_by": "worker-host-1234",
    "progress": {
      "pages": 12,
      "totalPages": 12,
      "chunks": 48,
      "heartbeatAt": "2026-07-23T16:13:14.000Z"
    },
    "timing": {
      "queuedMs": 1832,
      "runningMs": 932410,
      "totalMs": 934242,
      "heartbeatAt": "2026-07-23T16:13:14.000Z",
      "heartbeatAgeMs": null,
      "staleAfterMs": 120000,
      "isStale": false,
      "isRunning": false
    },
    "progressSummary": {
      "label": "12 of 12 pages",
      "percent": 100,
      "pageNumber": null,
```

```
      "processedPages": 12,
      "totalPages": 12,
      "chunks": 48,
      "generatedFiles": 1,
      "visionPages": 2
    }
  },
  "counts": {
    "chunks": 48,
    "embeddings": 48,
    "assets": 3
  },
  "requestId": "..."
}
```

`GET /api/v1/documents/{documentId}/ingestion-stats`

Scope: `documents:read`

Returns call-level PDF vision statistics for the document's latest `ingest` or `reprocess` job. Supply an optional `jobId` UUID to retrieve a specific older ingestion. A call is each actual HTTP request sent to a model provider, including transport retries and provider-compatibility fallbacks.

Response-time and pages-per-call standard deviations are population standard deviations over all observed calls. Failed HTTP responses, network failures, and timeouts count as failed calls and remain in the response-time distribution. Older ingestions report `partial` or `unavailable` coverage when they predate call-level observations.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/ingestion-stats" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "stats": {
    "documentId": "document-uuid",
    "operation": "pdf_direct_vision",
    "generatedAt": "2026-08-16T14:10:00.000Z",
    "job": {
      "id": "job-uuid",
      "type": "reprocess",
      "status": "succeeded",
      "stage": "complete",
      "queuedAt": "2026-08-16T14:00:00.000Z",
      "startedAt": "2026-08-16T14:00:02.000Z",
      "finishedAt": "2026-08-16T14:09:58.000Z"
    },
    "coverage": {
      "state": "complete",
      "observedCalls": 42,
      "unclassifiedCalls": 0,
      "note": null
    },
    "calls": {
      "total": 42,
      "observed": 42,
      "successful": 39,
      "failed": 3,
      "unclassified": 0,
      "successRate": 0.9286,
      "failureRate": 0.0714
    },
    "responseTimeMs": {
      "count": 42,
      "min": 8200,
      "max": 90012,
      "mean": 24118.38,
      "standardDeviation": 14782.41
    },
    "pagesPerCall": {
      "count": 42,
      "min": 1,
      "max": 5,
      "mean": 4.62,
      "standardDeviation": 1.04
    },
    "providers": [
      {
```

```
        "provider": "together",
        "model": "google/gemma-4-31b-it",
        "routeIds": ["route-2"],
        "calls": {
          "total": 22,
          "observed": 22,
          "successful": 21,
          "failed": 1,
          "unclassified": 0,
          "successRate": 0.9545,
          "failureRate": 0.0455
        },
        "responseTimeMs": {
          "count": 22,
          "min": 8200,
          "max": 61200,
          "mean": 19830.18,
          "standardDeviation": 10342.72
        },
        "pagesPerCall": {
          "count": 22,
          "min": 1,
          "max": 5,
          "mean": 4.73,
          "standardDeviation": 0.94
        }
      }
    ]
  },
  "requestId": "..."
}
```


GET /api/v1/documents/{documentId}/chunks

Scope: `documents:read`

Query params:

Name	Type	Notes
`page`	integer	Optional pagination page.
`pageSize`	integer	Optional pagination size.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/chunks?page=1&pageSize=50" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:


```
{
  "chunks": [
    {
      "id": "chunk-uuid",
      "document_id": "document-uuid",
      "page_id": "page-uuid",
      "asset_id": null,
      "page_number": 1,
      "chunk_index": 0,
      "chunk_type": "native_text",
      "chunk_text": "Text content...",
      "source_extraction_method": "native_pdf_text",
      "source_image_bucket": null,
      "source_image_path": null,
      "token_count": 300,
      "created_at": "2026-07-19T00:00:00.000Z",
      "updated_at": "2026-07-19T00:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 50,
    "total": 48,
    "totalPages": 1
  },
  "requestId": "..."
}
```

GET /api/v1/documents/{documentId}/text

Scope: `documents:read`

Downloads document text as Markdown.

If a generated source-assembly Markdown file exists for the document, that file is returned. Otherwise the API builds Markdown from stored chunks.

Request:

```
curl -L "$API_BASE/documents/$DOCUMENT_ID/text" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -o document.md
```

Response:

```
Content-Type: text/markdown; charset=utf-8
Content-Disposition: attachment; filename="source.md"
```

GET /api/v1/documents/{documentId}/download

Scope: `documents:read`

Downloads the original source file.

Request:

```
curl -L "$API_BASE/documents/$DOCUMENT_ID/download" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -o source.pdf
```

Response:

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="source.pdf"
```

POST /api/v1/documents/{documentId}/download-ticket

Scope: `documents:read`

Returns a signed Supabase Storage URL for the original source file. The URL expires in 60 to 300 seconds and should be opened immediately. This read-style POST does not require an idempotency key.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/download-ticket" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"expiresIn":120}'
```

Response:

```
{
  "download": {
    "url": "https://project.supabase.co/storage/v1/object/sign/...",
    "fileName": "source.pdf",
    "mimeType": "application/pdf",
    "byteSize": 15728640,
    "expiresIn": 120,
    "expiresAt": "2026-07-29T12:02:00.000Z"
  },
  "requestId": "request-uuid"
}
```

POST /api/v1/documents/{documentId}/markdown-download-ticket

Scope: `documents:read`

Uses the same request and response format as the original download ticket, but returns canonical Markdown. When no source-assembly Markdown object exists, the API assembles it, stores a versioned private cache in `generated-files`, records it as a `canonical_markdown` document file, and signs that object.

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/markdown-download-ticket" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"expiresIn":120}'
```

The `download` response contains the signed `url`, a `.md` `fileName`, `text/markdown; charset=utf-8` `mimeType`, `byteSize`, `expiresIn`, and `expiresAt`. The signed URL is temporary and does not require the bearer key.

`GET /api/v1/runpod/download/{jobId}`

Internal RunPod worker route. It accepts only the one-time bearer credential created for the matching embedding attempt. The response is canonical UTF-8 Markdown with its byte length and SHA-256 checksum. The route closes when the attempt completes, fails, activates, or expires. Normal API keys are not accepted.

`POST /api/v1/runpod/results`

Internal RunPod worker route. It accepts only the matching one-time bearer credential and requires an `Idempotency-Key`. Batch, completion, and failure payloads are validated against protocol version 1. Identical batch retries are acknowledged; conflicting retries fail the attempt. Result rows remain private until the Railway worker validates and atomically activates the complete set.

`POST /api/v1/documents/{documentId}/reprocess`

Scope: `documents:reprocess`

Idempotency required.

Queues reprocessing for an existing document. If an active queued/running job already exists, the endpoint returns the existing job with `alreadyQueued: true`.

Body:

Field	Type	Required	Notes
`processingDepth`	`fast` | `standard` | `deep`	No	Defaults to `standard`.
`visionCacheMode`	`refresh` | `reuse`	No	Defaults to `refresh`. Use `reuse` to keep prior page and embedded-image vision Markdown when reprocessing the same PDF.
`webhookUrl`	URL	No	Receives completion/failure callback.
`webhookSecret`	string	No	Up to 500 characters. Used for HMAC signature.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/reprocess" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: reprocess-$(uuidgen)" \
  -d '{
    "processingDepth": "deep",
    "visionCacheMode": "reuse",
    "webhookUrl": "https://example.com/bubble-rag/webhook",
    "webhookSecret": "replace-with-your-shared-secret"
  }'
```

Response status: `202`

```
{
  "document": {
    "id": "document-uuid",
    "tenant_id": "tenant-uuid",
    "status": "queued",
    "processing_depth": "deep"
  },
  "job": {
    "id": "job-uuid",
    "document_id": "document-uuid",
    "status": "queued",
    "stage": "registered",
    "queued_at": "2026-07-19T00:00:00.000Z"
  },
```

```
  "requestId": "..."
}
```

POST /api/v1/documents/{documentId}/embed

Scope: `documents:reprocess`

Idempotency required.

Queues an embedding-only backfill job for a document that is already `ready`. This does not rerun PDF parsing, OCR, vision analysis, Markdown assembly, or chunking. It only creates or refreshes missing `document_embeddings` rows for the active embedding version.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/embed" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: embed-$(uuidgen)"
```

Response status: `202`

```
{
  "document": {
    "id": "document-uuid",
    "tenant_id": "tenant-uuid",
    "status": "ready"
  },
  "job": {
    "id": "job-uuid",
    "document_id": "document-uuid",
    "status": "queued",
    "stage": "registered",
    "queued_at": "2026-07-19T00:00:00.000Z"
  },
  "requestId": "..."
}
```

POST /api/v1/documents/{documentId}/cancel

Scope: `documents:cancel`

Idempotency required.

Cancels the latest active queued/running ingestion or reprocess job for a document. Include `jobId` when cancelling a specific job returned by the status endpoint. The uploaded source document is not deleted; the document can be queued again after cancellation.

Body:

Field	Type	Required	Notes
`jobId`	UUID	No	Cancels that active job; omitted means latest active job.
`reason`	string	No	Up to 1000 characters. Stored on the cancelled job/document.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/cancel" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: cancel-$(uuidgen)" \
  -d '{
    "jobId": "job-uuid",
    "reason": "Cancelled because the job heartbeat went stale."
  }'
```

Response status: `202`

```
{
  "cancelled": true,
  "document": {
    "id": "document-uuid",
    "tenant_id": "tenant-uuid",
    "status": "failed",
    "last_error": "Cancelled because the job heartbeat went stale."
  },
```

```
  "job": {
    "id": "job-uuid",
    "document_id": "document-uuid",
    "status": "cancelled",
    "stage": "failed",
    "finished_at": "2026-07-23T00:00:00.000Z"
  },
  "requestId": "..."
}
```

`DELETE /api/v1/documents/{documentId}`

Scope: `documents:delete`

Idempotency required.

Deletes the document, related database rows through cascading constraints, source storage object references, page render storage objects, extracted asset objects, and generated files tied to that document.

Request:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID" \
  -X DELETE \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: delete-document-$(uuidgen)"
```

Response:

```
{
  "deleted": true,
  "document": {
    "id": "document-uuid",
    "file_name": "source.pdf",
    "title": "Source Agreement"
  },
  "removedStorageObjects": 5,
  "removedGeneratedFiles": 1,
  "requestId": "..."
}
```

Ingestion Webhooks

Document upload and reprocess can include webhook fields.

Upload form fields:

```
webhookUrl
webhookSecret
```

Reprocess JSON fields:

```
{
  "webhookUrl": "https://example.com/bubble-rag/webhook",
  "webhookSecret": "replace-with-your-shared-secret"
}
```

Webhook event names:

```
document.ingest.succeeded
document.ingest.failed
document.reprocess.succeeded
document.reprocess.failed
```

Webhook request headers:

Header	Notes
`Content-Type: application/json`	JSON payload.
`x-bubble-rag-event`	Event name.
`x-bubble-rag-delivery-id`	Unique delivery UUID.
`x-bubble-rag-signature`	Present only when `webhookSecret` is set. Format: `sha256=<hex-hmac>`.

Signature calculation:

```
hex_hmac_sha256(raw_request_body, webhookSecret)
```

Payload:

```
{
  "event": "document.ingest.succeeded",
  "tenantId": "tenant-uuid",
  "documentId": "document-uuid",
  "jobId": "job-uuid",
  "jobType": "ingest",
  "status": "succeeded",
  "errorMessage": null,
  "occurredAt": "2026-07-19T00:05:00.000Z"
}
```

Webhook delivery attempts are recorded in the database table `api_webhook_deliveries`.

Reference Libraries

Reference libraries are tenant-wide document collections outside every silo. A library contains projects; each project groups documents and defines one model-facing RAG tool. API usernames see the Libraries navigation only when their associated key has `libraries:read`.

`libraries:grant` is intentionally separate and is never inferred from an empty or legacy scope list. It changes which shared documents a silo can read.

Library document status, chunks, original files, Markdown, reprocessing, embedding, cancellation, and physical deletion use the existing `/api/v1/documents/{documentId}/...` endpoints. Those routes require `libraries:read` for read operations and `libraries:write` for mutations when the document has no silo. Silo document permissions do not grant access to library documents.

`GET /api/v1/libraries`

Scope: `libraries:read`. Lists libraries with standard `page` and `pageSize` pagination.

`POST /api/v1/libraries`

Scope: `libraries:write`. Idempotency required.

```
{
  "name": "Municipal reference material",
  "description": "Shared regulations and guidance used by municipal workflows."
}
```

`GET /api/v1/libraries/{libraryId}`

Scope: `libraries:read`. Returns one tenant-owned library.

`PATCH /api/v1/libraries/{libraryId}`

Scope: `libraries:write`. Idempotency required. Send `name`, `description`, or both.

`DELETE /api/v1/libraries/{libraryId}`

Scope: `libraries:write`. Idempotency required. Deletes the container, projects, grants, bindings, and memberships. Source document records and stored files are retained.

`GET /api/v1/libraries/{libraryId}/projects`

Scope: `libraries:read`. Lists projects inside one library with standard pagination.

`POST /api/v1/libraries/{libraryId}/projects`

Scope: `libraries:write`. Idempotency required.

```
{
  "toolName": "municipal_regulations",
  "displayName": "Municipal regulations",
  "description": "Search adopted municipal regulations, implementation guidance, and compliance
    requirements.",
  "enabled": true
}
```

`toolName` must be a unique lowercase tool identifier. The description is model-facing and must contain at least 30 characters.

GET /api/v1/libraries/{libraryId}/projects/{projectId}

Scope: `libraries:read`. Returns one project only when it belongs to the selected library and authenticated tenant.

PATCH /api/v1/libraries/{libraryId}/projects/{projectId}

Scope: `libraries:write`. Idempotency required. Accepts any combination of `toolName`, `displayName`, `description`, and `enabled`.

DELETE /api/v1/libraries/{libraryId}/projects/{projectId}

Scope: `libraries:write`. Idempotency required. Deletes the project, memberships, grants, and bindings without deleting its source documents.

GET /api/v1/libraries/{libraryId}/projects/{projectId}/documents

Scope: `libraries:read`. Lists project documents with status and download links. Supports standard pagination. Use each returned document ID with `GET /api/v1/documents/{documentId}/status` for heartbeat, page, chunk, and embedding progress.

POST /api/v1/libraries/{libraryId}/projects/{projectId}/documents

Scope: `libraries:write`. Idempotency required.

For a new upload, use the same multipart fields as silo ingestion: `file` or `fileUrl`, optional `fileName`, `title`, `mimeType`, `processingDepth`, `webhookUrl`, and `webhookSecret`. The document is created with a null silo, assigned to the selected library project, and queued immediately. A successful upload returns `202`.

To assign an existing library document to another project without re-ingesting it, send JSON:

```
{
  "documentId": "existing-library-document-uuid"
}
```

A silo document cannot be adopted into a library project.

DELETE /api/v1/libraries/{libraryId}/projects/{projectId}/documents/{documentId}

Scope: `libraries:write`. Idempotency required. Removes only this project membership. The document, chunks, embeddings, other project memberships, and stored files are retained.

POST /api/v1/libraries/{libraryId}/projects/{projectId}/search

Scope: `libraries:read`. Searches only ready documents assigned to the selected library project. It accepts the direct RAG search fields `query`, `topK`, `retrieve`, `documentId`, `minScore`, `searchMode`, `embeddingVersion`, `includeNeighborChunks`, `neighborChunkCount`, and `neighborChunkMaxChars`.

```
{
  "query": "What accessibility standards apply to sidewalks?",
  "topK": 20,
  "searchMode": "vector",
  "retrieve": "text"
}
```

GET /api/v1/libraries/{libraryId}/projects/{projectId}/access

Scope: libraries:grant. Returns the complete siloIds access list.

PUT /api/v1/libraries/{libraryId}/projects/{projectId}/access

Scope: libraries:grant. Idempotency required. Replaces the complete access list; omitted IDs lose access.

```
{
  "siloIds": ["silo-uuid-1", "silo-uuid-2"]
}
```

DELETE /api/v1/libraries/{libraryId}/projects/{projectId}/access/{siloId}

Scope: libraries:grant. Idempotency required. Revokes one silo. Existing bindings become ineffective because runtime tool resolution also requires a current access grant.

Template Groups

A Template Group defines one dynamically named model tool. Assigning the group to a silo makes that tool available to both Ask and Extraction. The model is instructed not to use it unless the user's instructions request a template. Specific template names use title search; general document purposes use the stored description vectors with title fallback.

GET /api/v1/template-groups

Scope: templates:read. Lists Template Groups and their stable IDs with standard page and pageSize pagination.

GET /api/v1/template-groups/{templateGroupId}

Scope: templates:read. Returns one tenant-owned Template Group.

PATCH /api/v1/template-groups/{templateGroupId}

Scope: templates:write. Idempotency required. Accepts any combination of name, toolName, toolDescription, and enabled.

GET /api/v1/template-groups/{templateGroupId}/templates

Scope: templates:read. Lists the active, non-deleted templates in the group with their stable IDs, latest ingestion status, latest job, asset count, review suggestions, and delete URL. Each latest-version review suggestion reports its stable ID, importance, category, saved Word page number when available, source location, issue, and recommended source-DOCX edit. The response also reports the number of findings ignored for the current ingestion. Supports standard page and pageSize pagination plus an optional query title filter.

Poll this endpoint while latestJob.status is queued or running. latestJob.stage is the current ingestion step and latestJob.lockedAt is the worker heartbeat; queuedAt, startedAt, finishedAt, attempts, and maxAttempts provide the remaining activity context. A descriptionEmbeddingStatus of awaiting_ingestion is expected during an active run because the description vector is queued only after ingestion has generated the detailed description.

An ingestion with no material warnings, unresolved items, or improvement suggestions finishes as completed. A needs_review version includes one or more actionable latestVersion.reviewSuggestions. Each suggestion may have several alternatives. Only alternatives with safeToImplement: true have an implementUrl. The latest version also reports its six reviewPasses, parent, source kind, review cycle, convergence status, and pinned contract digest.

POST /api/v1/template-groups/{templateGroupId}/templates/{templateId}/versions/{versionId}/review-findings/{findingId}/ignore

Scope: templates:write. Marks one open finding irrelevant for this immutable version. If no open findings remain, the version becomes completed. This does not edit the DOCX and therefore does not require idempotency.

POST /api/v1/template-groups/{templateGroupId}/templates/{templateId}/versions/{versionId}/review-findings/{findingId}/implement

Scope: `templates:write`. Idempotency required. Resolves the stored alternative server-side; callers cannot supply OOXML operations. The request must pin the exact source and compiled contract:

```
{
  "alternativeId": "set-scope-section",
  "expectedSourceSha256": "64-lowercase-hex-characters",
  "expectedContractDigest": "64-lowercase-hex-characters"
}
```

The server applies the validated edit to staged bytes, recompiles the DOCX, stores version N+1 with a parent/provenance link, records the action, and queues all six ingestion passes. It rejects stale versions, changed checksums/digests, unvalidated alternatives, and corrections beyond the review-cycle limit.

DELETE /api/v1/template-groups/{templateGroupId}/templates/{templateId}

Scope: `templates:write`. Idempotency required. Removes the template immediately from catalog and model-tool searches, cancels queued ingestion and embedding work, and frees its catalog name for a replacement upload. Immutable source versions and job history remain privately retained for reproducibility.

GET /api/v1/template-groups/{templateGroupId}/silos

Scope: `templates:grant`. Returns the complete `siloIds` assignment list.

PUT /api/v1/template-groups/{templateGroupId}/silos

Scope: `templates:grant`. Idempotency required. Replaces the complete list; omitted silo IDs lose access.

```
{
  "siloIds": ["silo-uuid-1", "silo-uuid-2"]
}
```

DELETE /api/v1/template-groups/{templateGroupId}/silos/{siloId}

Scope: `templates:grant`. Idempotency required. Removes one assignment.

GET /api/v1/federated-datasets

Scope: `federated:read`. Lists Federated Datasets and their stable IDs with standard `page` and `pageSize` pagination. Each entry reports the model tool name, description, enabled state, entry count, and assigned silo count. This is the call to make when you need a `datasetId` for the assignment routes.

POST /api/v1/federated-datasets

Scope: `federated:write`. Idempotency required. Creates a dataset definition. `toolName` is derived from `name` when omitted; it must be unique per tenant, 3-64 characters, and match `^[a-z][a-z0-9_]*$`.

```
{
  "name": "Grant Registry",
  "toolName": "grant_registry",
  "toolDescription": "One funded grant or capital project. Look up before writing any grant name
     or ID."
}
```

The definition is shared across silos. Its data is not: assigning it to two silos gives each silo its own entries and its own ID sequence.

GET /api/v1/federated-datasets/{datasetId}

Scope: `federated:read`. Returns one tenant-owned Federated Dataset.

PATCH /api/v1/federated-datasets/{datasetId}

Scope: `federated:write`. Idempotency required. Accepts any combination of `name`, `toolName`, `toolDescription`, `enabled`, `idPrefix`, `displayIdStrategy`, `displayIdPrefix`, `modelCanWrite`, and `modelCanDelete`. Editing a dataset never changes an entry.

DELETE /api/v1/federated-datasets/{datasetId}

Scope: `federated:write`. Idempotency required. **Destroys the definition and every silo's entries and aliases together.** There is no soft delete and no recovery. The response reports `deletedEntries` and `deletedSiloAssignments` so a caller can record what was removed. To stop a dataset being offered in a silo without losing that silo's data, unassign it instead.

GET /api/v1/federated-datasets/{datasetId}/silos

Scope: `federated:grant`. Returns the complete `siloIds` assignment list.

PUT /api/v1/federated-datasets/{datasetId}/silos

Scope: `federated:grant`. Idempotency required. Replaces the complete list. A silo omitted from the list is **unassigned, not cleared**: its entries and ID sequence are retained, and re-assigning the dataset restores them exactly.

```
{
  "siloIds": ["silo-uuid-1", "silo-uuid-2"]
}
```

DELETE /api/v1/federated-datasets/{datasetId}/silos/{siloId}

Scope: `federated:grant`. Idempotency required. Unassigns one silo. The tool stops resolving there; the silo's entries are retained.

DELETE /api/v1/federated-datasets/{datasetId}/silos/{siloId}/entries

Scope: `federated:write`. Idempotency required. **Clears that silo's entries for the dataset.** This is the destructive counterpart to unassignment, on a separate path and behind a different scope so the two cannot be confused. Returns `deletedEntries`.

GET /api/v1/silos/{siloId}/federated-datasets

Scope: `federated:read`. Lists the Federated Datasets currently assigned to one silo, with each one's model tool name, description, model write switches, and active entry count. Answers in one call what would otherwise be a full dataset list plus one assignment read per dataset.

GET /api/v1/silos/{siloId}/library-tools

Scope: `libraries:grant`. Lists silo-default library tool bindings. Optional query parameter `taskKey=ask|extraction` filters the response.

PUT /api/v1/silos/{siloId}/library-tools

Scope: `libraries:grant`. Idempotency required. Replaces the silo-default bindings for one task type. Every library project must already be granted to the silo.

```
{
  "taskKey": "ask",
  "bindings": [
    {
      "libraryProjectId": "library-project-uuid",
      "maxPolls": 3
    }
  ]
}
```

GET /api/v1/silos/{siloId}/projects/{projectId}/library-tools

Scope: `libraries:grant`. Lists project-specific bindings. These override the silo default for the same library tool when that silo project is selected.

PUT /api/v1/silos/{siloId}/projects/{projectId}/library-tools

Scope: `libraries:grant`. Idempotency required. Uses the same `taskKey` and `bindings` body as the silo-default endpoint, but stores the selection for one silo project.

RAG Settings

GET /api/v1/rag-settings

Scope: `silos:read`

Returns the tenant-level RAG ingestion, embedding, and direct-search settings. Chunking settings are used for newly ingested or reprocessed documents; existing chunks keep the chunking they were created with until a document is reprocessed. Use Configuration -> Embedding / Vector options to queue all ready documents after changing the model.

Response:


```
{
  "settings": {
    "id": "settings-uuid",
    "tenantId": "tenant-uuid",
    "chunkMaxTokens": 900,
    "chunkOverlapTokens": 120,
    "searchMinScore": 0.08,
    "neighborChunkCount": 1,
    "neighborChunkMaxChars": 2000,
    "embedding": {
      "provider": "ollama",
      "model": "qwen3-embedding:0.6b",
      "dimensions": 1024,
      "version": "ollama:qwen3-embedding:0.6b:1024:v1",
      "label": "Qwen3 Embedding 0.6B",
      "contextTokens": 32768
    },
    "updatedAt": "2026-07-20T10:00:00.000Z",
    "defaultsApplied": false,
    "limits": {
      "chunkMinTokens": 100,
      "chunkHardMaxTokens": 2000,
      "chunkOverlapHardMaxTokens": 500,
      "searchMinScoreMin": 0,
      "searchMinScoreMax": 1,
      "neighborChunkCountMin": 0,
      "neighborChunkCountMax": 10,
      "neighborChunkMaxCharsMin": 200,
      "neighborChunkMaxCharsMax": 50000
    }
  },
  "requestId": "..."
}
```


PUT /api/v1/rag-settings

Scope: `silos:write`

Idempotency: required

Updates the tenant-level chunking, embedding model, neighbor context, and direct-search settings. The API key decides the tenant, so clients do not need to send `tenantId`. Saving a model does not reprocess source files or automatically queue a backfill; queue it explicitly from Configuration after the model is saved.

Body:

Field	Type	Required	Notes
`chunkMaxTokens`	integer	Yes	100-2000. Hard max prevents oversized chunks.
`chunkOverlapTokens`	integer	Yes	0-500 and must be smaller than `chunkMaxTokens`.
`searchMinScore`	number	Yes	0-1. Default cutoff for direct RAG search filtering.
`neighborChunkCount`	integer	No	0-10. Number of chunks before and after each matched chunk to include in combined context.

Field	Type	Required	Notes
`neighborChunkMaxChars`	integer	No	200-50000. Per-neighbor text cap before the combined Markdown span is assembled.
`embeddingProvider`	string	No	`supabase`, `ollama`, or `openai`. Ollama embedding options run on the Railway worker's local Ollama instance.
`embeddingModel`	string	No	Includes `qwen3-embedding:0.6b`, `qwen3-embedding:4b`, the legacy Ollama options, `gte-small`, and the configured OpenAI option.
`embeddingDimensions`	integer	No	Must match the model: Qwen3 0.6B uses 1024 and Qwen3 4B uses 2560; legacy options use 384 or 768.

Request:

```
curl -sS "$API_BASE/rag-settings" \
  -X PUT \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: rag-settings-$(uuidgen)" \
  -d '{
    "chunkMaxTokens": 900,
    "chunkOverlapTokens": 120,
    "searchMinScore": 0.08,
    "neighborChunkCount": 2,
    "neighborChunkMaxChars": 12000,
    "embeddingProvider": "ollama",
    "embeddingModel": "qwen3-embedding:0.6b",
    "embeddingDimensions": 1024
  }'
```

`PATCH /api/v1/rag-settings`

Scope: `silos:write`. Idempotency: required.

Updates only `searchMinScore`, `neighborChunkCount`, and `neighborChunkMaxChars`. This is useful for search clients that must not alter the embedding model or document chunking configuration.

RAG Search

`POST /api/v1/silos/{siloId}/rag-search`

Scope: `documents:read`

Searches indexed chunks in one silo. This route is a read-style POST and does not require an idempotency key.

Body:

Field	Type	Required	Notes
`query`	string	Yes	1-2000 characters.
`topK`	integer	No	1-50. Defaults to `20`.
`retrieve`	`chunks` | `text` | `original`	No	Defaults to `text`.
`projectId`	UUID	No	Restricts search to documents assigned to this project/subset. If omitted, searches the whole silo.
`documentId`	UUID	No	Restricts search to one document.
`minScore`	number	No	0-1 cutoff. Defaults to the saved RAG setting.
`searchMode`	`keyword` | `vector` | `hybrid`	No	Defaults to `vector`. Vector mode uses pgvector cosine similarity. If query embeddings are unavailable, the response reports `embedding_unavailable`; select `hybrid` or `keyword` explicitly when keyword fallback is desired.
`embeddingVersion`	string	No	Optional embedding version filter, such as `ollama:nomic-embed-text:768:v1` or `supabase:gte-small:384:v1`.

Field	Type	Required	Notes
`includeNeighborChunks`	boolean	No	Defaults to `true`. Adds surrounding chunk context plus a combined context span for each match. When Markdown offsets are available, the combined span is sliced from the assembled Markdown file so repeated chunk overlap is not duplicated.
`neighborChunkCount`	integer	No	0-10. Defaults to the saved RAG setting. Number of chunks before and after the current match to include.
`neighborChunkMaxChars`	integer	No	200-50000. Defaults to the saved RAG setting, initially `2000` characters per context chunk.

Retrieve modes:

Mode	Behavior
`chunks`	Returns matched chunk snippets and shorter chunk text.
`text`	Returns matched snippets and longer text from each match.
`original`	Returns original-file metadata and an API download URL when available.

Request:

```
curl -sS "$API_BASE/silos/$SILO_ID/rag-search" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "query": "renewal date and termination notice",
    "topK": 5,
    "projectId": "project-uuid",
    "retrieve": "text",
    "minScore": 0.08,
    "searchMode": "vector",
    "includeNeighborChunks": true,
    "neighborChunkCount": 2
  }'
```

Response:


```
{
  "result": {
    "siloId": "silo-uuid",
    "query": "renewal date and termination notice",
    "retrieve": "text",
    "minScore": 0.08,
    "searchMode": "vector",
    "retrievalMode": "vector_rag_search",
    "embedding": {
      "attempted": true,
      "available": true,
      "provider": "ollama",
      "model": "nomic-embed-text",
      "version": "ollama:nomic-embed-text:768:v1",
      "latencyMs": 213,
      "error": null
    },
    "timing": {
      "settings": 18,
      "retrieval": 482,
      "context_chunks": 41,
      "markdown_context": 36,
      "context_assembly": 3,
      "response_assembly": 1,
      "retrievalPhases": {
        "readyDocumentsMs": 22,
        "keywordCandidatesMs": 0,
        "queryEmbeddingMs": 213,
        "vectorRpcMs": 84,
        "vectorHydrationMs": 17,
        "rankingMs": 2,
        "totalMs": 472
      },
      "totalMs": 581
    },
    "scoring": {
```

```
    "method": "vector_cosine_similarity_v1",
    "sorted": "score_desc",
    "scoreRange": {
      "min": 0,
      "max": 1
    }
  },
  "matches": [
    {
      "rank": 1,
      "score": 0.82,
      "scorePercent": 82,
      "rawScore": 24,
      "lexicalScore": 0.71,
      "lexicalRawScore": 24,
      "vectorScore": 0.86,
      "vectorDistance": 0.14,
      "embeddingProvider": "ollama",
      "embeddingModel": "nomic-embed-text",
      "embeddingVersion": "ollama:nomic-embed-text:768:v1",
      "termCoverage": 1,
      "matchedTerms": ["renewal", "date", "termination", "notice"],
      "matchedPhrases": ["termination notice"],
      "exactPhraseMatch": false,
      "chunkId": "chunk-uuid",
      "documentId": "document-uuid",
      "documentName": "client-a-master-services-agreement.pdf",
      "sourceDocumentName": "client-a-master-services-agreement.pdf",
      "documentTitle": "Client A Master Services Agreement",
      "fileName": "client-a-master-services-agreement.pdf",
      "sourceDocument": {
        "id": "document-uuid",
        "name": "client-a-master-services-agreement.pdf",
        "title": "Client A Master Services Agreement",
        "fileName": "client-a-master-services-agreement.pdf",
        "mimeType": "application/pdf",
        "pageCount": 12
      },
      "pageNumber": 4,
      "chunkType": "native_text",
      "sourceStartOffset": 18420,
      "sourceEndOffset": 26385,
      "snippet": "The agreement renews automatically...",
      "chunkContext": {
        "previous": {
          "chunkId": "previous-chunk-uuid",
          "documentId": "document-uuid",
          "pageNumber": 3,
          "chunkIndex": 7,
          "chunkType": "native_text",
          "text": "The preceding section discusses renewal notice delivery...",
          "overlapTrimmedChars": 286,
          "sourceStartOffset": 10710,
          "sourceEndOffset": 18420
        },
        "current": {
          "chunkId": "chunk-uuid",
          "documentId": "document-uuid",
          "pageNumber": 4,
          "chunkIndex": 8,
          "chunkType": "native_text",
          "text": "The agreement renews automatically unless either party gives written
            notice...",
          "overlapTrimmedChars": 0,
          "sourceStartOffset": 18420,
          "sourceEndOffset": 26385
        },
        "next": {
          "chunkId": "next-chunk-uuid",
          "documentId": "document-uuid",
          "pageNumber": 4,
          "chunkIndex": 9,
          "chunkType": "native_text",
          "text": "The following paragraph defines the termination effective date...",
          "overlapTrimmedChars": 302,
          "sourceStartOffset": 26385,
          "sourceEndOffset": 33720
        },
        "combined": {
          "documentId": "document-uuid",
          "chunkIds": [
            "previous-chunk-uuid",
            "chunk-uuid",
            "next-chunk-uuid"
```

```
            ],
            "sourceStartOffset": 10710,
            "sourceEndOffset": 33720,
            "text": "The surrounding Markdown section without duplicated overlap...",
            "source": "markdown_offsets"
          }
        },
        "text": "The agreement renews automatically unless either party gives written notice..."
      }
    ]
  },
  "requestId": "..."
}
```


Successful searches return the same phase durations in the `Server-Timing` response header. A failed search returns `error.details.phase` and `error.details.timing`, allowing clients and API usage logs to distinguish settings, embedding, vector RPC, neighboring chunks, canonical Markdown, context assembly, and response assembly failures. API audit persistence has a separate bounded `api_audit` server-timing metric and does not hold a completed search response open.

Important source-citation guidance:

- Use `sourceDocument.name` when showing the human-readable source document name.
- Use `sourceDocument.id` or top-level `documentId` when linking, deduplicating, downloading, or storing references.
- Several documents can have the same file name, so keep the ID with the name.

Example source display:

```
Source: client-a-master-services-agreement.pdf (document-uuid), page 4
```

Ask

`POST /api/v1/silos/{siloId}/ask`

Scope: `rag:ask`

Idempotency required.

Asks a grounded question in the selected silo. The endpoint presents the request and scoped tool descriptions to the configured Ask model first. The model decides whether it needs silo evidence, then issues focused RAG or source inspection calls before composing the answer or building the requested file.

The ordinary JSON endpoint and streaming endpoint use the same bounded Ask agent loop. The request reaches the model before retrieval, and the model may request one enabled `rag_search`, `list_files`, or `get_document_markdown` operation at a time. Railway executes the tool in the selected silo/project scope, returns the tool result to the model, and continues until the model produces a final answer or reaches the Ask task's configured tool-request limit. The default is 64 and the Configuration page accepts 1-250. `rag_search` has a separate configurable 0-100 request ceiling, defaulting to 8. The model never supplies a silo or project ID. For non-DOCX, non-template requests, an unavailable provider may fall back to a retrieval-only grounded preview.

`get_document_markdown` is an internal model tool, not a public API endpoint. The model supplies a document ID returned by RAG search and receives exact canonical Markdown. Larger files include `nextOffset` and `complete`, allowing lossless sequential retrieval without silently truncating the document. Railway rejects documents outside the selected silo/project scope.

Every Ask model call receives a freshly generated UTC timestamp plus the Ask task's configured reference location and IANA time zone. This includes each follow-up tool-loop call, so relative dates and daylight-saving rules are evaluated against the current request time. API callers do not send these fields.

Body:

Field	Type	Required	Notes
`question`	string	Yes	1 character up to the configured Ask Model Task maximum; default 100,000 and absolute ceiling 1,000,000.
`topK`	integer	No	1-50. Defaults to the configured Ask task Top K.
`projectId`	UUID	No	Restricts Ask to a project/subset inside the silo.

Field	Type	Required	Notes
`documentId`	UUID	No	Restricts Ask to one ready document in the silo.
`chunkTypes`	array	No	Optional list of chunk types to use.
`searchMode`	`keyword` | `vector` | `hybrid`	No	Overrides the configured Ask task RAG search mode. New or unconfigured Ask tasks default to `vector`.
`embeddingVersion`	string	No	Optional embedding version filter.
`convertRequestToModelTools`	boolean	No	When true, the configured Ask model first rewrites the request for the tools and swarm routes actually available in the selected silo. Defaults to `false`.
`outputFile.enabled`	boolean	No	When true, stores the answer as a generated file.
`outputFile.format`	`txt` | `csv` | `md` | `json` | `docx`	No	Defaults to `txt`.
`outputFile.fileName`	string	No	1-180 characters. Extension is added when missing.
`webhookUrl`	URL	No	Receives every `model.status.updated` event. Must use HTTP or HTTPS.
`webhookSecret`	string	No	Up to 500 characters. Signs callback bodies with HMAC-SHA256 and requires `webhookUrl`.

`outputFile.enabled` requires the Ask model task to have tools enabled and include the `write_file` tool.

When `convertRequestToModelTools` is true, Railway performs an Ask-only preflight model call. The original question remains in history and reruns; the rewritten request drives the normal Ask coordinator. The preflight receives a compact source-document inventory, uses lightweight reasoning, and returns a structured, deduplicated execution plan with focused RAG objectives and bounded swarm/cache assignments when useful. Configure the editable conversion policy under **Configuration -> Model Tasks -> agentic instructions for Ask queries**. Conversion progress and the rewritten request are available in the Ask event stream and `usage.agentic_request_conversion`.

For Markdown and text output, Railway preserves an explicit `Source` or `Sources` section already present in the model's final answer and does not add a second source appendix. When the answer does not include an explicit source section, Railway appends one normalized source list from the retrieved chunk metadata. JSON and CSV output continue to expose sources as structured fields.

Request:

```
curl -sS "$API_BASE/silos/$SILO_ID/ask" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: ask-$(uuidgen)" \
  -d '{
    "question": "What is the renewal date and where is it stated?",
    "convertRequestToModelTools": true,
    "topK": 8,
    "webhookUrl": "https://example.com/bubble-rag/ask-status",
    "webhookSecret": "replace-with-your-shared-secret",
    "outputFile": {
      "enabled": true,
      "format": "md",
      "fileName": "renewal-answer.md"
    }
  }'
```

Response:

```
{
  "answer": "The agreement renews automatically on ... [1]",
  "confidence": "medium",
  "sources": [
    {
      "documentId": "document-uuid",
      "documentName": "client-a-master-services-agreement.pdf",
      "documentTitle": "Client A Master Services Agreement",
      "documentFileName": "client-a-master-services-agreement.pdf",
      "sourceDocument": {
        "id": "document-uuid",
        "name": "client-a-master-services-agreement.pdf",
        "title": "Client A Master Services Agreement",
        "fileName": "client-a-master-services-agreement.pdf",
```

```
        "mimeType": "application/pdf",
        "pageCount": 12
      },
      "chunkId": "chunk-uuid",
      "pageNumber": 4,
      "chunkIndex": 2,
      "chunkType": "native_text",
      "snippet": "The agreement renews automatically...",
      "sourceImageBucket": null,
      "sourceImagePath": null,
      "boundingBox": null,
      "score": 14
    }
  ],
  "usage": {
    "retrievedChunks": 8,
    "candidateChunks": 100,
    "retrievalMode": "vector_rag_search",
    "searchMode": "vector",
    "embedding": {
      "attempted": true,
      "available": true,
      "provider": "ollama",
      "model": "nomic-embed-text",
      "version": "ollama:nomic-embed-text:768:v1",
      "latencyMs": 213,
      "error": null
    },
    "providerCalls": 1,
    "modelCallAttempted": true,
    "modelLatencyMs": 1964,
    "reasoningEffort": "medium",
    "reasoningCharacters": 1840,
    "toolCalls": [
      {
        "tool": "rag_search",
        "query": "renewal date notice period",
        "matchCount": 8
      }
    ],
    "modelError": null
  },
  "model": {
    "provider": "ollama",
    "name": "llama3.2-vision"
  },
  "questionLog": {
    "id": "question-log-uuid",
    "created_at": "2026-07-19T00:00:00.000Z"
  },
  "generatedFile": {
    "id": "generated-file-uuid",
    "fileName": "renewal-answer.md"
  },
  "askRun": {
    "id": "question-log-uuid",
    "status": "succeeded",
    "statusUrl": "/api/v1/ask-runs/question-log-uuid",
    "startedAt": "2026-07-31T12:00:00.000Z",
    "finishedAt": "2026-07-31T12:00:08.000Z",
    "webhookConfigured": true,
    "webhookSigned": true
  },
  "requestId": "..."
}
```


If no indexed chunks are available, the endpoint returns a low-confidence answer explaining that no indexed chunks are available.

`POST /api/v1/silos/{siloId}/ask/stream`

Scope: `rag:ask`

Idempotency required.

Starts the same Ask workflow as the ordinary endpoint but returns `application/x-ndjson`. Every line is one complete JSON event. Do not parse the whole response as one JSON document. Read a line, parse that line, update the UI, and continue until the connection closes.

The request body is identical to `POST /api/v1/silos/{siloId}/ask`.

Request:

```
curl -N "$API_BASE/silos/$SILO_ID/ask/stream" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Accept: application/x-ndjson" \
  -H "Idempotency-Key: ask-stream-$(uuidgen)" \
  -d '{
    "question": "What is the renewal date and where is it stated?",
    "projectId": "project-uuid",
    "searchMode": "vector",
    "topK": 20
  }'
```

The response headers include:

```
Content-Type: application/x-ndjson; charset=utf-8
X-Ask-Run-Id: question-log-uuid
Location: /api/v1/ask-runs/question-log-uuid
Cache-Control: no-cache, no-transform
```

Each event has this envelope:

```
{
  "runId": "question-log-uuid",
  "sequence": 7,
  "type": "tool.completed",
  "payload": {},
  "createdAt": "2026-07-31T12:00:04.000Z"
}
```

Event types:

Event type	Meaning
`ask.started`	Durable Ask run created and execution started.
`retrieval.started`	Initial silo/project-scoped RAG retrieval started.
`retrieval.completed`	Retrieval counts, search mode, retrieval mode, and embedding status are available.
`retrieval.expansion.required`	A broad Ask attempted to finish before its configured number of distinct follow-up searches; the payload reports completed, required, and remaining searches.
`model.call.started`	A provider call started. `call` identifies the loop iteration.
`reasoning.delta`	Provider-visible reasoning information. See provider differences below.
`model.output.delta`	Incremental model output. Intermediate output may be a JSON tool request.
`model.call.completed`	Provider call timing, usage, reasoning/output sizes, and requested tool are available.
`model.call.failed`	The provider call failed.
`model.status.updated`	The model published a user-visible milestone. The payload may include phase, state, task key/label, and current/total step counts.
`tool.started`	Railway accepted an actual tool request from the model.
`tool.completed`	Railway completed the tool and reports its summary plus a bounded result preview.
`tool.failed`	Railway could not execute the requested tool.
`answer.completed`	The model produced a final answer rather than another tool request.
`generated_file.prepared`	Existing same-name output was checked/deleted before writing replacement output.
`generated_file.completed`	The requested generated file was stored.
`ask.completed`	Terminal success event. `payload.result` is the complete ordinary Ask response object.
`ask.failed`	Terminal failure event with standardized `code`, `message`, and HTTP-equivalent `status`.
`ask.cancelled`	Terminal cancellation event after the active provider call and swarm workers receive cancellation.
`ask.replayed`	The Idempotency-Key already belongs to an existing run; poll the returned `statusUrl`.
`swarm.started`	A `swarm_task` dispatch started; payload includes worker count and configured routes.

Event type	Meaning
`swarm.worker.queued`	One focused worker was queued with its cache slot.
`swarm.worker.started`	One worker acquired a route; payload includes provider, model, reasoning effort, and route ID.
`swarm.worker.tool.started`	A worker started an application tool call.
`swarm.worker.tool.completed`	A worker tool call completed and includes a bounded summary.
`swarm.worker.tool.failed`	A worker tool call failed.
`swarm.worker.completed`	A worker completed; payload includes cache name, provider-call count, result preview, and runtime.
`swarm.worker.failed`	A worker failed while other independent workers may continue.
`swarm.worker.cancelled`	A worker was stopped by parent cancellation.
`swarm.completed`	All workers settled; payload includes completed/failed/cancelled counts and wall time.
`swarm.failed`	Every worker failed, so the coordinator receives no successful worker slot.
`swarm.cancelled`	Parent cancellation stopped the dispatch.

Example abbreviated stream:


```
{"runId":"question-log-uuid","sequence":1,"type":"ask.started","payload":{"searchMode":"vector",
  "topK":20},"createdAt":"2026-07-31T12:00:00.000Z"}
{"runId":"question-log-uuid","sequence":2,"type":"retrieval.started","payload":{"searchMode":"ve
  ctor","topK":20},"createdAt":"2026-07-31T12:00:00.100Z"}
{"runId":"question-log-uuid","sequence":3,"type":"retrieval.completed","payload":{"retrievedChun
  ks":20,"candidateChunks":160},"createdAt":"2026-07-31T12:00:00.800Z"}
{"runId":"question-log-uuid","sequence":4,"type":"reasoning.delta","payload":{"delta":"I should
  verify the notice period against another section."},"createdAt":"2026-07-31T12:00:01.100Z"}
{"runId":"question-log-uuid","sequence":5,"type":"tool.started","payload":{"tool":"rag_search","
  arguments":{"query":"renewal notice period"}},"createdAt":"2026-07-31T12:00:02.000Z"}
{"runId":"question-log-uuid","sequence":6,"type":"tool.completed","payload":{"tool":"rag_search"
  ,"query":"renewal notice period","matchCount":8},"createdAt":"2026-07-31T12:00:02.500Z"}
{"runId":"question-log-uuid","sequence":7,"type":"ask.completed","payload":{"result":{"answer":"
  The agreement renews ... [1]","confidence":"medium","sources":[],"askRun":{"id":"question-log-
  uuid","status":"succeeded"}}},"createdAt":"2026-07-31T12:00:08.000Z"}
```


When the model requests complete source context, streamed tool events identify `get_document_markdown`. The full Markdown remains in the private model/tool conversation; the event journal stores a bounded preview and completion metadata rather than duplicating the entire document.

Provider-visible reasoning differs by provider:

- Ollama thinking-capable models stream their `message.thinking` content when supported by the selected model and configured effort.
- OpenAI uses Responses API reasoning-summary events. This is a provider summary, not private raw chain-of-thought. Models that do not support the reasoning-summary option are retried without it and continue streaming model output.
- Modular forwards provider-exposed reasoning deltas when the selected model includes them in its OpenAI-compatible streaming response.
- Tool events are produced by Railway from actual execution. They do not rely on the model narrating which tool it believes it used.

The stream deliberately continues server-side execution if the client closes the connection. Retain `X-Ask-Run-Id` or the `runId` from the first event and poll the status endpoint.

`GET /api/v1/ask-runs`

Scope: `rag:ask`

Lists persisted Ask runs newest first. Use this endpoint when the original stream disconnected before Bubble saved the run ID, or to build an Ask queue and history view. List items are compact summaries; use each item's `statusUrl` to retrieve its full answer, generated files, usage, and event journal.

Query params:

Name	Required	Default	Notes
`siloId`	Yes	-	UUID of the silo whose Ask history is listed.
`projectId`	No	-	Restricts results to one project in the silo.
`status`	No	-	`queued`, `running`, `succeeded`, `failed`, or `cancelled`.
`createdAfter`	No	-	Inclusive ISO 8601 lower timestamp bound.
`createdBefore`	No	-	Inclusive ISO 8601 upper timestamp bound.
`page`	No	`1`	Result page.
`pageSize`	No	`20`	Items per page, from 1 through 100.

Request:

```
curl -sS
  "$API_BASE/ask-runs?siloId=$SILO_ID&projectId=$PROJECT_ID&status=succeeded&page=1&pageSize=50"
   \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "askRuns": [
    {
      "id": "question-log-uuid",
      "status": "succeeded",
      "terminal": true,
      "currentStage": "completed",
      "heartbeatAt": "2026-08-01T12:00:08.000Z",
      "siloId": "silo-uuid",
      "projectId": "project-uuid",
      "documentId": null,
      "question": "What is the latest population?",
      "answerPreview": "The latest cited population is ...",
      "model": { "provider": "ollama", "name": "qwen3" },
      "sourceCount": 10,
      "startedAt": "2026-08-01T12:00:00.000Z",
      "finishedAt": "2026-08-01T12:00:08.000Z",
      "elapsedMs": 8000,
      "webhookConfigured": true,
      "webhookSigned": true,
      "statusUrl": "/api/v1/ask-runs/question-log-uuid"
    }
  ],
  "filters": {
    "siloId": "silo-uuid",
    "projectId": "project-uuid",
    "status": "succeeded",
    "createdAfter": null,
    "createdBefore": null
  },
  "pagination": { "page": 1, "pageSize": 50, "total": 1, "totalPages": 1 },
  "requestId": "..."
}
```

To recover an Ask whose ID was not saved, list the relevant silo and optional project, identify the run by `question`, `startedAt`, and status, then call its `statusUrl`.

`DELETE /api/v1/ask-runs`

Scope: `rag:ask`

Clears persisted terminal Ask history. This operation never deletes `queued` or `running` runs. Associated `ask_run_events` rows are deleted by cascade; generated files are retained and their Ask link is set to null.

Headers:

- `Authorization: Bearer <api-key>`
- `Idempotency-Key: <new-unique-value>`

Query params:

Name	Required	Default	Notes
`siloId`	Yes	-	UUID of the silo to clean.
`projectId`	No	-	Restricts cleanup to one project.
`status`	No	All terminal statuses	`succeeded`, `failed`, or `cancelled` only.
`before`	No	Request time	Delete terminal runs created at or before this timestamp.
`confirm`	Yes	-	Must be exactly `true`.

Request:

```
curl -sS -X DELETE \
  "$API_BASE/ask-runs?siloId=$SILO_ID&projectId=$PROJECT_ID&before=2026-08-01T00%3A00%3A00.000Z&
    confirm=true" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: clear-ask-runs-$(date +%s)"
```

Response:

```
{
  "cleared": {
    "siloId": "silo-uuid",
    "projectId": "project-uuid",
    "before": "2026-08-01T00:00:00.000Z",
    "statuses": ["succeeded", "failed", "cancelled"],
    "deletedRuns": 12,
    "skippedActiveRuns": 1,
    "eventJournalsDeleted": true,
    "conversationsDeleted": 12,
    "generatedFilesRetained": true
  },
  "requestId": "..."
}
```

Filters belong in the query string. Do not send a JSON body with either the list or cleanup call.

`GET /api/v1/ask-runs/{runId}`

Scope: `rag:ask`

Returns durable Ask status, current stage, heartbeat, timing, answer, sources, generated files, provider/model usage, and persisted events. The Ask run ID is the same UUID as `questionLog.id`.

Query params:

Name	Default	Range	Notes
`afterSequence`	`0`	0+	Return only events whose sequence is greater than this value.
`eventLimit`	`100`	1-500	Maximum events returned in this request.

Request:

```
curl -sS "$API_BASE/ask-runs/$ASK_RUN_ID?afterSequence=7&eventLimit=200" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "askRun": {
    "id": "question-log-uuid",
    "status": "running",
    "terminal": false,
    "currentStage": "model_running",
    "heartbeatAt": "2026-07-31T12:00:05.000Z",
    "siloId": "silo-uuid",
    "projectId": "project-uuid",
    "documentId": null,
    "question": "What is the renewal date?",
    "answer": null,
    "confidence": null,
    "model": {
      "provider": "ollama",
```

```
      "name": "qwen3"
    },
    "sources": [],
    "generatedFiles": [],
    "usage": {
      "lifecycle_status": "running",
      "current_stage": "model_running"
    },
    "startedAt": "2026-07-31T12:00:00.000Z",
    "finishedAt": null,
    "elapsedMs": 5100,
    "webhookConfigured": true,
    "webhookSigned": true,
    "statusUrl": "/api/v1/ask-runs/question-log-uuid"
  },
  "events": [],
  "eventPage": {
    "afterSequence": 7,
    "eventLimit": 200,
    "returned": 0,
    "hasMore": false,
    "nextAfterSequence": 7
  },
  "requestId": "..."
}
```


Polling rules:

1. Save the run ID immediately from `X-Ask-Run-Id` or the first event.
2. Poll every 1-3 seconds while `terminal` is `false`.
3. Pass the previous `nextAfterSequence` value to receive only newer events.
4. Stop polling when `terminal` is `true`.
5. For `succeeded`, use `answer`, `sources`, and `generatedFiles` from `askRun`. For `failed`, inspect `usage.error_code`, `usage.model_error`, and the terminal `ask.failed` event.

`POST /api/v1/ask-runs/{runId}/cancel`

Scope: `rag:ask`

Cancels a queued or running Ask. The run status becomes `cancelled`, the active provider request is aborted, OpenAI OAuth turns are interrupted, and every queued or running swarm worker receives the same cancellation signal. A client closing the NDJSON stream by itself still does not cancel the run; call this endpoint explicitly.

```
curl -sS -X POST "$API_BASE/ask-runs/$ASK_RUN_ID/cancel" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: unique-client-generated-key" \
  -d '{}'
```

The response is idempotent for an already-terminal run and reports its current status. Poll `GET /api/v1/ask-runs/{runId}` to retrieve the terminal event and the last persisted swarm telemetry.

Extractions

Standalone Extraction is synchronous. Railway performs scoped RAG retrieval, runs the configured Extraction model/tool loop, writes the generated file, and persists the extraction run and result before returning HTTP `200`. This call can therefore remain open for the configured model timeout. Use a stable idempotency key when retrying after a client timeout.

Saved multi-step Tasks use a different execution model: `POST /api/v1/tasks/{taskId}/run` returns `202` and runs the extraction steps in the background one by one. Use Task runs when the caller needs a durable queued workflow containing one or more extraction steps.

`GET /api/v1/silos/{siloId}/extractions`

Scope: `rag:extract`

Query params:

Name	Type	Notes
`page`	integer	Optional pagination page.
`pageSize`	integer	Optional pagination size.
`projectId`	UUID	Optional project/subset filter.
`status`	string	Optional extraction run status filter.

Request:

```
curl -sS "$API_BASE/silos/$SILO_ID/extractions?page=1&pageSize=20" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "extractionRuns": [
    {
      "id": "extraction-run-uuid",
      "tenant_id": "tenant-uuid",
      "project_id": "project-uuid",
      "silo_id": "silo-uuid",
      "document_id": "document-uuid",
      "template_id": null,
      "status": "queued",
      "model_config_snapshot": {},
      "input_schema": {},
      "error_message": null,
      "started_at": null,
      "finished_at": null,
      "created_at": "2026-07-19T00:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 1,
    "totalPages": 1
  },
  "requestId": "..."
}
```

`POST /api/v1/silos/{siloId}/extractions`

Scope: `rag:extract`

Idempotency required.

Body:

Field	Type	Required	Notes
`projectId`	UUID	No	Restrict extraction to a project/subset inside the silo.
`documentId`	UUID	No	Restrict extraction to one document in the silo.
`request`	string	Conditional	Extraction instructions up to the configured Extraction Model Task maximum; default 100,000 and absolute ceiling 1,000,000. Preferred field.
`templateId`	UUID	No	Reserved for template-based extraction; not applied by the current executor.
`customSchema`	object	No	Compatibility wrapper. `customSchema.request` is used only when top-level `request` is omitted. Other members are not currently applied.
`topK`	integer	No	1-50. Defaults to the configured Extraction task Top K.
`chunkTypes`	array	No	Optional list of chunk types to use.
`searchMode`	`keyword` | `vector` | `hybrid`	No	Overrides the configured Extraction task RAG search mode. New or unconfigured Extraction tasks default to `vector`.

Field	Type	Required	Notes
`embeddingVersion`	string	No	Optional embedding version filter.
`outputFormat`	`json` | `csv` | `markdown` | `docx`	No	Defaults to `json`.
`processingDepth`	`fast` | `standard` | `deep`	No	Accepted for request compatibility. Current standalone execution uses the saved Extraction model configuration.
`outputFile.enabled`	boolean	No	Set true when supplying `outputFile.format` or `fileName`. A generated file is created for every successful standalone extraction.
`outputFile.format`	`txt` | `csv` | `md` | `json` | `docx`	No	Overrides `outputFormat` when `outputFile.enabled` is true.
`outputFile.fileName`	string	No	1-180 characters. Used only when `outputFile.enabled` is true; otherwise Railway generates a name.
`webhookUrl`	URL	No	Receives every `model.status.updated` event. Must use HTTP or HTTPS.
`webhookSecret`	string	No	Up to 500 characters. Signs callback bodies with HMAC-SHA256 and requires `webhookUrl`.

Provide either top-level `request` or `customSchema.request`. Prefer the top-level field for new integrations.

Request:

```
curl -sS "$API_BASE/silos/$SILO_ID/extractions" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: extraction-$(uuidgen)" \
  -d '{
    "request": "Extract the effective date, renewal date, and termination notice period. Return
      unknown for absent values and include source references.",
    "documentId": "document-uuid",
    "outputFormat": "json",
    "webhookUrl": "https://example.com/bubble-rag/extraction-status",
    "webhookSecret": "replace-with-your-shared-secret",
    "outputFile": {
      "enabled": true,
      "format": "json",
      "fileName": "contract-fields.json"
    }
}'
```

Response status: `200`

```
{
  "extractionRun": {
    "id": "extraction-run-uuid",
    "status": "succeeded",
    "created_at": "2026-08-01T12:00:00.000Z",
    "started_at": "2026-08-01T12:00:00.000Z",
    "finished_at": "2026-08-01T12:00:08.000Z"
  },
  "extractionResult": {
    "id": "extraction-result-uuid",
    "created_at": "2026-08-01T12:00:08.000Z"
  },
  "generatedFile": {
    "id": "generated-file-uuid",
    "silo_id": "silo-uuid",
    "project_id": "project-uuid",
    "extraction_run_id": "extraction-run-uuid",
    "file_name": "contract-fields.json",
    "output_format": "json",
    "mime_type": "application/json",
    "byte_size": 842
  },
  "sourceCount": 8,
  "modelCallAttempted": true,
  "modelLatencyMs": 7421,
  "modelCallCount": 2,
  "modelToolCalls": [],
  "outputNormalization": "none",
  "modelError": null,
  "webhookConfigured": true,
  "webhookSigned": true,
  "preview": "{\n  \"effective_date\": ...\n}",
```

```
  "queued": false,
  "requestId": "..."
}
```

The Extraction task must have tools enabled and allow `write_file`. A request with no scoped RAG sources returns `422 extraction_no_rag_sources`; a disabled file tool returns `422 write_file_tool_disabled`; provider or output-format failures return a standardized error while retaining a failed extraction run for diagnosis. Use `GET /api/v1/extractions/{extractionId}` later to reopen the persisted run, structured result, and associated generated files.

`GET /api/v1/extractions/{extractionId}`

Scope: `rag:extract`

Fetches an extraction run, its result row if available, and generated files tied to the extraction run.

Request:

```
curl -sS "$API_BASE/extractions/$EXTRACTION_ID" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "extractionRun": {
    "id": "extraction-run-uuid",
    "status": "succeeded",
    "document_id": "document-uuid",
    "error_message": null,
    "progress": {
      "message": "Validating the extracted dates.",
      "phase": "Validation",
      "currentStep": 3,
      "totalSteps": 4
    },
    "webhookConfigured": true,
    "webhookSigned": true
  },
  "result": {
    "id": "extraction-result-uuid",
    "result_json": {
      "effective_date": "2026-01-01"
    },
    "result_markdown": null,
    "result_csv": null,
    "missing_fields": [],
    "source_references": [],
    "confidence": "medium",
    "created_at": "2026-07-19T00:00:00.000Z"
  },
  "generatedFiles": [
    {
      "id": "generated-file-uuid",
      "file_name": "contract-fields.json",
      "output_format": "json",
      "content_preview": "{...}"
    }
  ],
  "requestId": "..."
}
```

Ask and standalone Extraction model-status webhooks

When an Ask or standalone Extraction request includes `webhookUrl`, every successful `model_status` tool call sends an HTTP `POST`. The callback includes the original request text so a receiver can display the update without making a second status request.

```
{
  "event": "model.status.updated",
  "deliveryId": "delivery-uuid",
  "tenantId": "tenant-uuid",
  "runType": "ask",
  "askRunId": "question-log-uuid",
  "extractionRunId": null,
  "siloId": "silo-uuid",
  "projectId": "project-uuid",
  "documentId": null,
  "request": "Prepare the monthly manager report.",
  "modelStatus": {
    "message": "Collecting current grant and project updates.",
```

```
    "phase": "RAG retrieval",
    "state": "working",
    "currentStep": 2,
    "totalSteps": 6,
    "taskKey": "grant-research",
    "taskLabel": "Grant research"
  },
  "occurredAt": "2026-08-24T12:00:00.000Z"
}
```

For Extraction, `runType` is `extraction`, `extractionRunId` is populated, and `askRunId` is `null`. The same callback headers and HMAC verification contract documented under Task webhooks apply. `webhookConfigured` and `webhookSigned` are returned without exposing the URL or secret. Delivery attempts are written to `api_webhook_deliveries` against `question_log_id` or `extraction_run_id`. Webhook failures do not fail the Ask or Extraction run.

Tasks

Tasks are reusable extraction workflows. A saved Task is not tied to a silo. A Task may start with no extraction items while it is being drafted. A Task run is tied to one selected silo and executes the saved extraction items sequentially. If a Task has five extraction items, a successful run creates five generated files in that silo. If a Task has no items, the run completes successfully without creating generated files.

Each extraction item uses the configured Extraction model task, including its configured RAG search mode and retrieval top K, plus the existing `write_file` generated-file path. A Task step only stores the step name, extraction request, output file name, and output file type. If an item specifies a file name that already exists in the target silo, the extraction tool overwrites it by deleting the existing generated file first.

`GET /api/v1/tasks`

Scope: `rag:extract`

Lists saved Task definitions. Supports standard `page` and `pageSize` pagination.

```
curl -sS "$API_BASE/tasks?page=1&pageSize=20" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

The response contains `tasks` and `pagination`. Each Task uses the same shape as `GET /api/v1/tasks/{taskId}`, including its ordered `items` array.

`POST /api/v1/tasks`

Scope: `rag:extract`

Idempotency required.

Body:

Field	Type	Required	Notes
`name`	string	Yes	Human-readable Task name.
`description`	string	No	Optional notes for the Task definition.
`isActive`	boolean	No	Defaults to `true`.
`items`	array	No	Optional ordered extraction steps. 0-25 items. Prefer `/items` for sequence editing.
`items[].name`	string	Yes	Step label.
`items[].request`	string	Yes	Extraction instructions for this step.
`items[].outputFileName`	string	Yes	Generated file name for this step.
`items[].outputFormat`	`csv` | `md` | `json`	No	Defaults to `json`. Use `md` for Markdown.

Example:

```
curl -sS "$API_BASE/tasks" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: task-$(uuidgen)" \
  -d '{
    "name": "Planning Report Extracts",
    "description": "Create the standard files from a selected silo.",
    "items": [
      {
        "name": "Project list",
        "request": "Extract all named projects, locations, dates, and costs.",
        "outputFileName": "projects.json",
        "outputFormat": "json"
      },
      {
        "name": "Sidewalk findings",
        "request": "Extract every sidewalk-related project or goal with source context.",
        "outputFileName": "sidewalk-findings.md",
        "outputFormat": "md"
      }
    ]
  }'
```

To create an empty draft Task, omit `items`:

```
curl -sS "$API_BASE/tasks" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: task-$(uuidgen)" \
  -d '{
    "name": "Draft extraction workflow",
    "description": "Work in progress"
  }'
```

GET /api/v1/tasks/{taskId}

Scope: `rag:extract`

Returns one saved Task and its ordered extraction steps.

```
curl -sS "$API_BASE/tasks/$TASK_ID" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "task": {
    "id": "task-uuid",
    "name": "Planning Report Extracts",
    "description": "Create the standard files from a selected silo.",
    "isActive": true,
    "items": [
      {
        "id": "task-item-uuid",
        "position": 1,
        "name": "Project list",
        "request": "Extract all named projects, locations, dates, and costs.",
        "outputFileName": "projects.json",
        "outputFormat": "json"
      }
    ]
  },
  "requestId": "..."
}
```

PUT /api/v1/tasks/{taskId}

Scope: `rag:extract`

Idempotency required.

Updates the Task name, description, active state, and optionally its complete step sequence. Use the dedicated `/items` endpoint when only editing or reordering steps.

```
curl -sS "$API_BASE/tasks/$TASK_ID" \
  -X PUT \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
```

```
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: task-update-$(uuidgen)" \
  -d '{
    "name": "Planning Report Extracts v2",
    "description": "Updated workflow description.",
    "isActive": true
  }'
```

DELETE /api/v1/tasks/{taskId}

Scope: `rag:extract`

Idempotency required.

Deletes the saved Task definition. This does not delete generated files that were created by completed Task runs.

```
curl -sS "$API_BASE/tasks/$TASK_ID" \
  -X DELETE \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: task-delete-$(uuidgen)"
```

Response:

```
{
  "deleted": true,
  "taskId": "task-uuid",
  "requestId": "..."
}
```

PUT /api/v1/tasks/{taskId}/items

Scope: `rag:extract`

Idempotency required.

Replaces the saved extraction sequence for a Task. Pass an empty `items` array to clear the sequence while keeping the Task name and description.

```
curl -sS "$API_BASE/tasks/$TASK_ID/items" \
  -X PUT \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: task-items-$(uuidgen)" \
  -d '{
    "items": [
      {
        "name": "Project list",
        "request": "Extract all named projects, locations, dates, and costs.",
        "outputFileName": "projects.json",
        "outputFormat": "json"
      }
    ]
  }'
```

POST /api/v1/tasks/{taskId}/run

Scope: `rag:extract`

Idempotency required.

Runs a saved Task against one silo. Add `projectId` to run the Task against a project/subset; omit it to run against the complete silo. Empty Tasks complete successfully and create no generated files.

Body:

Field	Type	Required	Notes
`siloId`	UUID	Yes	Silo searched by every extraction step.
`projectId`	UUID	No	Restricts every step to one project/subset.
`webhookUrl`	URL	No	Receives Task-run lifecycle callbacks and every model-status update. Must use HTTP or HTTPS.

Field	Type	Required	Notes
`webhookSecret`	string	No	Up to 500 characters. Signs callback bodies with HMAC-SHA256; requires `webhookUrl`.
`use_previous_generated_reference`	boolean	No	Defaults to `false`. When `true`, each step reads the existing output file it will overwrite and supplies that content to the Extraction model as historical reference material.

```
curl -sS "$API_BASE/tasks/$TASK_ID/run" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: task-run-$(uuidgen)" \
  -d '{
    "siloId": "silo-uuid",
    "projectId": "project-uuid",
    "use_previous_generated_reference": true,
    "webhookUrl": "https://example.com/bubble-rag/task-status",
    "webhookSecret": "replace-with-your-shared-secret"
  }'
```

When `use_previous_generated_reference` is `true`, the runner looks in the selected silo for the newest generated file with the task item's output name and format. If found, its complete content is read before overwrite and added to that item's model context as the previous version of the output. The model is instructed to:

- reuse an old identifier for a current entry only when current RAG sources support the same entity;
- never recycle an old identifier for a different entity or meaning;
- treat the old file as historical reference data rather than current source evidence; and
- omit entries no longer found in current sources unless the item request asks for historical/progress reporting, in which case the original identifier and entry may be retained with a `not found` status or the equivalent requested status.

If no matching output exists, the item runs normally without a previous reference. Passing `false` or omitting the field preserves the existing behavior.

Response status: `202`

```
{
  "taskRun": {
    "id": "task-run-uuid",
    "taskId": "task-uuid",
    "taskName": "Planning Report Extracts",
    "siloId": "silo-uuid",
    "siloName": "Georgetown test",
    "projectId": "project-uuid",
    "projectName": "Sidewalk Projects",
    "status": "queued",
    "currentItemPosition": null,
    "totalItems": 2,
    "completedItems": 0,
    "failedItems": 0,
    "webhookConfigured": true,
    "webhookSigned": true,
    "items": [
      {
        "id": "task-run-item-uuid",
        "position": 1,
        "name": "Project list",
        "outputFileName": "projects.json",
        "status": "queued",
        "extractionRunId": null,
        "generatedFileId": null
      }
    ]
  },
  "queued": true,
  "requestId": "..."
}
```

Task-run webhook callbacks

When `webhookUrl` is present, the Task runner sends an HTTP `POST` with a JSON body for each lifecycle event:

- `task.run.started` after the run enters `running`

- `task.run.progress` after each extraction item succeeds
- `model.status.updated` whenever the active item's model calls the `model_status` tool
- one terminal event: `task.run.succeeded`, `task.run.failed`, or `task.run.cancelled`

Example callback body:

```
{
  "event": "task.run.progress",
  "deliveryId": "delivery-uuid",
  "tenantId": "tenant-uuid",
  "taskId": "task-uuid",
  "taskRunId": "task-run-uuid",
  "siloId": "silo-uuid",
  "projectId": "project-uuid",
  "status": "running",
  "currentItemPosition": 1,
  "totalItems": 2,
  "completedItems": 1,
  "failedItems": 0,
  "errorMessage": null,
  "startedAt": "2026-08-09T12:00:00.000Z",
  "finishedAt": null,
  "item": {
    "id": "task-run-item-uuid",
    "position": 1,
    "name": "Project list",
    "request": "Extract all named projects, locations, dates, and costs.",
    "status": "succeeded",
    "extractionRunId": "extraction-run-uuid",
    "generatedFileId": "generated-file-uuid",
    "errorMessage": null
  },
  "request": "Extract all named projects, locations, dates, and costs.",
  "modelStatus": null,
  "occurredAt": "2026-08-09T12:00:08.000Z"
}
```

Payload fields:

Field	Type	Notes
`event`	string	A `task.run.*` lifecycle event or `model.status.updated`.
`deliveryId`	UUID	Unique to this delivery attempt; suitable as an idempotency key for the receiver.
`tenantId`	UUID	Tenant that owns the run.
`taskId`	UUID or `null`	Saved Task definition.
`taskRunId`	UUID	Authoritative Task-run identifier to use for status reconciliation.
`siloId`	UUID or `null`	Silo selected for the run.
`projectId`	UUID or `null`	Optional project/subset selected for the run.
`status`	string	Run status at the time the event was created.
`currentItemPosition`	integer/null	One-based position of the current or most recently completed item.
`totalItems`	integer	Number of saved items captured for the run.
`completedItems`	integer	Successfully completed item count.
`failedItems`	integer	Failed item count.
`errorMessage`	string/null	Run-level failure or cancellation message when present.
`startedAt`	timestamp/null	Run start time.
`finishedAt`	timestamp/null	Terminal completion time.
`item`	object/null	Completed item for `progress`, failing item for `failed` when known, otherwise `null`.
`request`	string/null	Active Task-item extraction request for model-status and item events.
`modelStatus`	object/null	Validated model-status message, phase, state, task identity, and step counts.
`occurredAt`	timestamp	UTC time at which the callback payload was created.

Delivery request headers:

Header	Present	Value
`content-type`	Always	`application/json`
`x-bubble-rag-event`	Always	Same lifecycle event name as the body `event` field.
`x-bubble-rag-delivery-id`	Always	Same UUID as the body `deliveryId` field.
`x-bubble-rag-signature`	Signed runs only	`sha256=<hex-hmac>` computed over the exact raw UTF-8 request body.

Verify a signed callback against the raw request body before parsing its JSON. Delivery attempts are recorded, and a callback failure does not change the Task-run result. An unsigned run omits `x-bubble-rag-signature`. The signing secret is encrypted at rest and is never returned by Task-run APIs.

Registering the webhook. `webhookUrl` and `webhookSecret` are per run, not per Task. They are supplied on `POST /api/v1/tasks/{taskId}/run` and stored on that Task run only, so every run that needs callbacks must send them again. Confirm registration from the `202` response fields `taskRun.webhookConfigured` and `taskRun.webhookSigned`; `GET /api/v1/task-runs/{runId}` reports the same two booleans. Neither the URL nor the secret is ever returned.

Validation applied when the run is created:

- `webhookUrl` is limited to 2048 characters and must use `http:` or `https:`. Anything else fails with `Webhook URL must use HTTP or HTTPS.`
- `webhookSecret` is limited to 500 characters and requires `webhookUrl`. Sending a secret alone fails with `A webhook URL is required when a signing secret is set.`
- Signing requires `ADMIN_SESSION_SECRET` to be configured on the web service. Without it, a run that sets `webhookSecret` fails with `500 task_run_create_failed`. The secret is encrypted with AES-256-GCM before it is stored.

Event sequence. Every run with a webhook receives `task.run.started`, zero or more `model.status.updated` events, and exactly one terminal event.

Event	Sent when	
`task.run.started`	The run moves from `queued` to `running`.	`null`
`task.run.progress`	An extraction item finishes successfully.	The completed item.
`model.status.updated`	The active model publishes a user-visible progress update.	The active item and request.
`task.run.succeeded`	All items completed. Empty Tasks emit this right after start.	`null`
`task.run.failed`	An item or the run failed. The run stops here.	The failing item, when known.
`task.run.cancelled`	`POST /api/v1/task-runs/{runId}/cancel` was accepted.	`null`

A failed item produces `task.run.failed`, not a `task.run.progress` event.

Delivery semantics.

- **One attempt per event. There are no retries.** A timeout or non-2xx response is logged and the run continues unchanged.
- Each delivery times out after 10 seconds, then the request is aborted.
- Deliveries are sent inline and in order, and the run waits for each one, so a slow endpoint slows the whole Task run by up to 10 seconds per event. Return `200` immediately and do your processing asynchronously.
- Any non-2xx response is recorded as failed with `Webhook returned HTTP <status>.`
- Every attempt is written to `api_webhook_deliveries` against `extraction_task_run_id`, including `event_type`, `status`, `status_code`, `error_message`, the full `request_body`, and the first 2000 characters of your `response_body`. Do not echo anything sensitive in your response body.
- `deliveryId` is a fresh UUID per event. Use it, or `taskRunId` plus `event` plus `item.position`, to make your handler idempotent.

Verifying the signature. Compute HMAC-SHA256 over the exact raw request body and compare against `x-bubble-rag-signature` in constant time. Parse the JSON only after the comparison succeeds.

```
import { createHmac, timingSafeEqual } from "node:crypto";

function isValidTaskWebhook(rawBody, signatureHeader, secret) {
  const expected = `sha256=${createHmac("sha256", secret).update(rawBody).digest("hex")}`;
  const a = Buffer.from(expected);
  const b = Buffer.from(signatureHeader ?? "");

  return a.length === b.length && timingSafeEqual(a, b);
}
```

Reconciling. Because there are no retries, treat callbacks as an optimisation rather than a guarantee. If a terminal event has not arrived when expected, poll `GET /api/v1/task-runs/{runId}` for the authoritative status and per-item `generatedFileId` values.

GET /api/v1/task-runs

Scope: `rag:extract`

Query params:

Name	Type	Notes
`page`	integer	Optional pagination page.
`pageSize`	integer	Optional pagination size.
`siloId`	UUID	Optional silo filter.
`projectId`	UUID	Optional project/subset filter.
`taskId`	UUID	Optional Task definition filter.
`status`	string	Optional run status filter.

Response:

```
{
  "taskRuns": [
    {
      "id": "task-run-uuid",
      "taskId": "task-uuid",
      "siloId": "silo-uuid",
      "projectId": "project-uuid",
      "status": "succeeded",
      "totalItems": 2,
      "completedItems": 2,
      "failedItems": 0
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 1,
    "totalPages": 1,
    "hasMore": false
  },
  "requestId": "..."
}
```

GET /api/v1/task-runs/{runId}

Scope: `rag:extract`

Returns the Task run plus ordered item records. Each item reports `status`, `extractionRunId`, `generatedFileId`, `sourceCount`, `modelLatencyMs`, `preview`, and any `errorMessage`. Download generated output with `GET /api/v1/generated-files/{fileId}/download`.

```
curl -sS "$API_BASE/task-runs/$TASK_RUN_ID" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "taskRun": {
    "id": "task-run-uuid",
    "taskId": "task-uuid",
    "taskName": "Planning Report Extracts",
    "siloId": "silo-uuid",
    "projectId": "project-uuid",
    "status": "running",
    "currentItemPosition": 2,
    "totalItems": 3,
    "completedItems": 1,
    "failedItems": 0,
    "errorMessage": null,
    "progress": { "status": "running" },
    "items": [
      {
        "id": "task-run-item-uuid",
        "position": 1,
        "name": "Project list",
        "outputFileName": "projects.json",
        "outputFormat": "json",
        "status": "succeeded",
        "extractionRunId": "extraction-run-uuid",
        "generatedFileId": "generated-file-uuid",
        "sourceCount": 8,
        "modelLatencyMs": 7421,
        "errorMessage": null
      }
    ]
  },
  "requestId": "..."
}
```

`POST /api/v1/task-runs/{runId}/cancel`

Scope: `rag:extract`

Idempotency required.

Cancels a queued or running Task run. Queued steps are marked `cancelled`; the active provider request receives an abort signal, OpenAI OAuth receives a turn interrupt, and queued or running swarm workers are stopped. The runner also checks the durable cancellation flag before another extraction item can start.

```
curl -sS "$API_BASE/task-runs/$TASK_RUN_ID/cancel" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: task-cancel-$(uuidgen)" \
  -d '{}'
```

Generated Files

Generated files are private Supabase Storage objects tracked in the `generated_files` table. They are associated with a silo and may reference a source document, Ask question log, extraction run, task key, provider, and model.

`GET /api/v1/silos/{siloId}/generated-files`

Scope: `generated-files:read`

Query params:

Name	Type	Notes
`page`	integer	Optional pagination page.
`pageSize`	integer	Optional pagination size.
`projectId`	UUID	Optional project/subset filter.
`taskKey`	`vision` | `ask` | `extraction` | `pdf_assembly`	Optional task filter.
`outputFormat`	`txt` | `csv` | `md` | `json` | `docx`	Optional format filter.
`temporary`	`true` | `false`	Optional temporary-file filter.

Request:

```
curl -sS
  "$API_BASE/silos/$SILO_ID/generated-files?taskKey=ask&outputFormat=md&page=1&pageSize=20" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Response:

```
{
  "generatedFiles": [
    {
      "id": "generated-file-uuid",
      "tenant_id": "tenant-uuid",
      "project_id": "project-uuid",
      "silo_id": "silo-uuid",
      "document_id": "document-uuid",
      "question_log_id": "question-log-uuid",
      "extraction_run_id": null,
      "task_key": "ask",
      "file_name": "renewal-answer.md",
      "output_format": "md",
      "mime_type": "text/markdown",
      "byte_size": 2048,
      "is_temporary": false,
      "generated_by_provider": "ollama",
      "generated_by_model": "model-name",
      "tool_name": "write_file",
      "content_preview": "# RAG Ask Output...",
      "metadata": {},
      "created_at": "2026-07-19T00:00:00.000Z",
      "updated_at": "2026-07-19T00:00:00.000Z",
      "deleteUrl": "/api/v1/generated-files/generated-file-uuid",
      "downloadUrl": "/api/v1/generated-files/generated-file-uuid/download",
      "downloadTicketUrl": "/api/v1/generated-files/generated-file-uuid/download-ticket",
      "jsonUrl": null
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 1,
    "totalPages": 1
  },
  "requestId": "..."
}
```

For an entry whose `output_format` is `json`, `jsonUrl` contains `/api/v1/generated-files/{fileId}/json`. Other formats return `null`.

GET /api/v1/generated-files/{fileId}/json

Scope: `generated-files:read`

Returns the stored JSON content as the complete response body. There is no `data`, `file`, or `requestId` wrapper, so an object, array, scalar, or `null` is returned directly. The current request ID is available in the `X-Request-Id` response header.

```
curl -sS "$API_BASE/generated-files/$FILE_ID/json" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Example response:

```
{
  "population": 7429,
  "year": 2019,
  "source": "2021 Georgetown Comprehensive Plan"
}
```

If the generated file has any format other than `json`, the endpoint returns HTTP `415` with `error.code` set to `generated_file_wrong_type`. If the file is marked as JSON but its stored content cannot be parsed, it returns HTTP `422` with `error.code` set to `generated_file_json_invalid`.

GET /api/v1/generated-files/{fileId}/download

Scope: `generated-files:read`

Downloads a generated file.

Request:

```
curl -L "$API_BASE/generated-files/$FILE_ID/download" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -o generated-output.md
```

Response:

```
Content-Type: text/markdown
Content-Disposition: attachment; filename="renewal-answer.md"
```

`POST /api/v1/generated-files/{fileId}/download-ticket`

Scope: `generated-files:read`

Returns a signed, short-lived URL for a generated file. Send `{}` to use the 120-second default, or set `expiresIn` from 60 through 300.

```
curl -sS "$API_BASE/generated-files/$FILE_ID/download-ticket" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"expiresIn":120}'
```

Response:

```
{
  "download": {
    "url": "https://project.supabase.co/storage/v1/object/sign/...",
    "fileName": "renewal-answer.md",
    "mimeType": "text/markdown",
    "byteSize": 1842,
    "expiresIn": 120,
    "expiresAt": "2026-08-01T12:02:00.000Z"
  },
  "requestId": "..."
}
```

`DELETE /api/v1/generated-files/{fileId}`

Scope: `generated-files:delete`

Idempotency required.

Deletes the generated file row and the private Supabase Storage object that backs it. This is useful for cleanup of extraction output, Ask-generated artifacts, and temporary PDF/vision assembly files.

Request:

```
curl -sS -X DELETE "$API_BASE/generated-files/$FILE_ID" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: delete-generated-file-$(uuidgen)"
```

Response:

```
{
  "deleted": true,
  "file": {
    "id": "generated-file-uuid",
    "fileName": "renewal-answer.md"
  },
  "removedFiles": 1,
  "removedStorageObjects": 1,
  "requestId": "..."
}
```

Typical Workflows

Workflow 1: Create a Silo, Upload a Document, Wait Until Ready, Ask a Question

1. Create a silo:

```
curl -sS "$API_BASE/silos" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
```

```
  -H "Idempotency-Key: create-silo-$(uuidgen)" \
  -d '{"name":"Client A"}'
```

1. Upload a document:

```
curl -sS "$API_BASE/silos/$SILO_ID/documents" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: upload-$(uuidgen)" \
  -F "file=@./source.pdf" \
  -F "processingDepth=standard"
```

1. Poll status:

```
curl -sS "$API_BASE/documents/$DOCUMENT_ID/status" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Wait until:

```
{
  "document": {
    "status": "ready"
  }
}
```

1. Ask a question:

```
curl -sS "$API_BASE/silos/$SILO_ID/ask" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: ask-$(uuidgen)" \
  -d '{"question":"Summarize the document in three bullets.","topK":8}'
```

Workflow 2: RAG Search Then Download a Source Document

1. Search:

```
curl -sS "$API_BASE/silos/$SILO_ID/rag-search" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"query":"payment terms","topK":3,"retrieve":"original"}'
```

1. Read the first match:

```
{
  "documentId": "document-uuid",
  "sourceDocument": {
    "id": "document-uuid",
    "name": "source.pdf"
  },
  "originalFile": {
    "downloadUrl": "/api/v1/documents/document-uuid/download"
  }
}
```

1. Download:

```
curl -L "$API_BASE/documents/$DOCUMENT_ID/download" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -o source.pdf
```

Workflow 3: Ask and Save the Answer as a Generated File

```
curl -sS "$API_BASE/silos/$SILO_ID/ask" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: ask-file-$(uuidgen)" \
  -d '{
    "question": "Extract the renewal date, payment terms, and termination notice requirement.",
    "topK": 20,
    "outputFile": {
      "enabled": true,
      "format": "csv",
      "fileName": "contract-summary.csv"
    }
  }'
```

Then download the generated file:

```
curl -L "$API_BASE/generated-files/$FILE_ID/download" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -o contract-summary.csv
```

Workflow 4: Upload With a Webhook Callback

```
curl -sS "$API_BASE/silos/$SILO_ID/documents" \
  -X POST \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY" \
  -H "Idempotency-Key: upload-webhook-$(uuidgen)" \
  -F "file=@./large-document.pdf" \
  -F "processingDepth=deep" \
  -F "webhookUrl=https://example.com/bubble-rag/webhook" \
  -F "webhookSecret=replace-with-your-shared-secret"
```

Your webhook receiver should:

1. Read the raw request body.
2. Compute `sha256=<hex_hmac_sha256(raw_body, webhookSecret)>`.
3. Compare it to `x-bubble-rag-signature` using constant-time comparison.
4. Use `event`, `documentId`, `jobId`, and `status` to update your external system.

Bubble API Connector Notes

For Bubble:

- Store the API key in Bubble's private API Connector authentication settings, not in client-visible fields.
- Use `Authorization: Bearer YOUR_KEY` as a shared header.
- For JSON endpoints, set `Content-Type: application/json`.
- For file ingestion, configure a multipart/form-data call with a file parameter named `file`.
- Add an `Idempotency-Key` header on create/upload/delete/cancel/reprocess/ask/extraction calls. A Bubble-generated UUID or unique workflow ID is ideal.
- Define the streamed Ask call as response type **Stream** and use `/silos/[silo_id]/ask/stream`.
- Treat each received line as one JSON event. Map or parse `payload.delta` for live reasoning/model output and inspect `type` to distinguish tool and terminal events.
- Save `X-Ask-Run-Id` when Bubble exposes response headers. Also save `runId` from the first parsed stream event so status recovery does not depend on header access.
- On a client error or page navigation, call `GET /ask-runs/[run_id]` and poll until `askRun.terminal` is `yes`.
- Save returned IDs:
- `silo.id`
- `document.id`
- `job.id`
- `questionLog.id`
- `askRun.id`
- `generatedFile.id`
- `extractionRun.id`
- For source citations, store both `sourceDocument.name` and `sourceDocument.id`.

Operational Notes

`GET /api/v1/usage-logs`

Every authenticated public API request writes an `api_usage_logs` row with:

- Tenant ID.
- API key ID.
- Request ID.
- Method.
- Path.
- Required scope.
- Status code.
- Latency.
- Error code when applicable.
- Query metadata.
- Whether an idempotency key was present.

These logs are intended for auditing API usage and troubleshooting external integrations.

Read them through the public API:

```
GET /api/v1/usage-logs
```

Scope: `api-usage:read`

Query parameters:

Name	Notes
`page`	Pagination page.
`pageSize`	Pagination size from 1 through 100.
`scope`	`current` for the calling key, or `tenant` for all tenant keys.
`statusCode`	Exact HTTP status code from 100 through 599.
`method`	Exact HTTP method such as `GET`, `POST`, or `DELETE`.
`errorCode`	Exact standardized API error code.
`path`	Case-insensitive partial request-path match.
`requestId`	Exact Railway request ID.
`createdAfter`	Inclusive ISO 8601 timestamp.
`createdBefore`	Inclusive ISO 8601 timestamp.

Sensitive metadata fields such as authorization, tokens, keys, secrets, passwords, signed URLs, and webhook values are recursively redacted before the response is returned.

```
curl -sS "$API_BASE/usage-logs?scope=current&statusCode=500&pageSize=60" \
  -H "Authorization: Bearer $BUBBLE_RAG_API_KEY"
```

Idempotency Storage

Idempotent JSON responses are stored in `api_idempotency_keys`. Replayed responses are returned as JSON with the original saved status code. Streaming Ask stores the created run identity instead of the potentially large stream body. Replaying that request returns an `ask.replayed` event and never starts a duplicate model run.

Security

- API keys grant tenant-wide access according to their scopes.
- Silo-level API key restrictions are not implemented yet.
- Keep API keys in server-side or private integration configuration.
- Do not expose API keys in browser JavaScript, Bubble client workflows, public logs, screenshots, or generated files.
- Rotate/delete API keys under Railway admin **Configuration -> API functionality -> API keys** when no longer needed.

Sleep/Wake Behavior

The Railway web service is configured with `sleepApplication: true`. A sleeping service may take a short moment to wake on the next request.

Current Limitations

- RAG search supports keyword, vector, and hybrid modes. Vector/hybrid ranking requires `document_embeddings` rows for the active embedding version; otherwise requests fall back to keyword retrieval and report the embedding error in usage/scoring metadata.
- Ask and Extraction use separately saved configurations over one shared scoped tool runtime. Each supports a configurable 1-250 model-requested tool rounds, defaulting to 64, plus an independent 0-100 RAG-search ceiling, defaulting to 8, and receives current UTC/reference-time-zone context on every provider call.
- Ollama thinking output depends on the selected model. OpenAI reasoning-summary output depends on Responses API model support. Modular reasoning output depends on the selected model's OpenAI-compatible stream. Tool and status events remain available even when a model does not expose reasoning information.
- Saved Task runs currently execute sequential extraction steps from the Railway web service background runner. If very large chained Tasks become common, this can move to a dedicated worker queue using the same task-run tables.
- API keys are scoped by capability, not by individual silo.
- The public API does not currently expose API-key management endpoints; manage keys under Railway admin **Configuration -> API functionality -> API keys**.

Minimal Client Example

Node.js example:

```
const API_BASE = "https://web-production-3bf65.up.railway.app/api/v1";
const API_KEY = process.env.BUBBLE_RAG_API_KEY;

async function api(path, options = {}) {
  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers: {
      authorization: `Bearer ${API_KEY}`,
      ...(options.body instanceof FormData
        ? {}
        : { "content-type": "application/json" }),
      ...(options.headers ?? {}),
    },
  });

  if (!response.ok) {
    const error = await response.json().catch(() => null);
    throw new Error(error?.error?.message ?? `HTTP ${response.status}`);
  }

  const contentType = response.headers.get("content-type") ?? "";
  return contentType.includes("application/json")
    ? response.json()
    : response.arrayBuffer();
}

const { silos } = await api("/silos");
const siloId = silos[0].id;
```

```
const answer = await api(`/silos/${siloId}/ask`, {
  method: "POST",
  headers: {
    "idempotency-key": crypto.randomUUID(),
  },
  body: JSON.stringify({
    question: "What are the main topics in this silo?",
    topK: 8,
  }),
});

console.log(answer.answer);
console.log(answer.sources.map((source) => source.sourceDocument));
```

Streaming Ask example:

```
async function streamAsk(siloId, question) {
  const response = await fetch(`${API_BASE}/silos/${siloId}/ask/stream`, {
    method: "POST",
    headers: {
      authorization: `Bearer ${API_KEY}`,
      "content-type": "application/json",
      "idempotency-key": crypto.randomUUID(),
    },
    body: JSON.stringify({ question, searchMode: "vector", topK: 20 }),
  });

  if (!response.ok || !response.body) {
    throw new Error(`Streaming Ask failed with HTTP ${response.status}`);
  }

  const runId = response.headers.get("x-ask-run-id");
  const reader = response.body.getReader();
  const decoder = new TextDecoder();
  let buffer = "";

  while (true) {
    const { done, value } = await reader.read();
    buffer += decoder.decode(value, { stream: !done });
    const lines = buffer.split(/\r?\n/);
    buffer = lines.pop() ?? "";

    for (const line of lines) {
      if (!line.trim()) continue;
      const event = JSON.parse(line);
      console.log(event.type, event.payload);

      if (event.type === "ask.completed") {
        return event.payload.result;
      }
    }

    if (done) break;
  }

  if (runId) {
    return api(`/ask-runs/${runId}`);
  }

  throw new Error("Ask stream closed before a terminal event.");
}
```